RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

CS23432 Software Construction

LABORATORY OBSERVATION

Name	:	Kevin Immanuel S
Year	:	IInd Year
Branch	:	Computer Science & Engineering
Register No	:	240701261
Semester	:	IV
Academic Year	:	2025-2026

RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

NAME: **KEVIN IMMANUEL S** REGISTER NO.**240701261**

ACADEMIC YEAR 2025-26 SEMESTER- IV
BRANCH: CSE

This Certification is the Bonafide record of work done by the above student in the CS23432 Software Construction Laboratory during the year 2025-2026.

Submitted for the Practical Examination held on

Signature of Faculty -in Charge

Internal Examiner External Examiner

CrimeTrack

Secure Crime Reporting Portal

Table of Contents

Experiment-1

Study of Azure Cloud Services

Microsoft Azure is a cloud computing platform offering over 200 products and services. The CrimeTrack portal leverages Azure's cloud ecosystem to deliver a scalable, secure, and government-grade web application. Understanding the three primary service models is essential before selecting the appropriate hosting and deployment strategy.

Cloud Service Models — Overview

Model	Full Name	Managed By Provider	Managed By User	CrimeTrack Usage
SaaS	Software as a Service	Everything	Data & Users	Azure DevOps (Project Management)
PaaS	Platform as a Service	OS, Runtime, Middleware	App Code & Data	Azure App Service (Backend API)
IaaS	Infrastructure as a Service	Networking, Storage, VMs	OS, Runtime, App, Data	Azure Virtual Machines (DB Server)

1. SaaS — Software as a Service

SaaS delivers fully managed software over the internet. Users access applications via a browser without managing infrastructure, middleware, or runtime environments. The provider handles everything from hardware to software updates.

Key Characteristics

- Accessible from any browser — no local installation required
- Subscription-based pricing (per user/month)
- Automatic updates and patch management by the provider
- Multi-tenant architecture — shared infrastructure across clients
- Examples: Microsoft 365, Salesforce, GitHub, Azure DevOps

CrimeTrack — SaaS Usage: Azure DevOps

Azure DevOps is the SaaS tool used to manage the CrimeTrack project. It provides:

- Boards — Kanban and Scrum backlogs for sprint planning
- Repos — Git-based version control for React frontend and Node.js backend
- Pipelines — CI/CD automation for build and deployment
- Test Plans — Manual and automated test case management
- Artifacts — Package management for npm and Docker images

2. PaaS — Platform as a Service

PaaS provides a managed platform on which developers deploy and run applications without managing the underlying operating system, runtime, or infrastructure. The cloud provider handles scaling, load balancing, and OS patches.

Key Characteristics

- Managed runtime environment (Node.js, Python, Java, .NET)
- Auto-scaling based on traffic load
- Built-in DevOps integration with CI/CD pipelines
- Developer focuses only on application code and data
- Examples: Azure App Service, Heroku, Google App Engine

CrimeTrack — PaaS Usage: Azure App Service

The CrimeTrack Express.js backend API is deployed on Azure App Service (PaaS):

- Runtime: Node.js 18 LTS on Linux container
- API hosted at: https://crimetrack-api.azurewebsites.net/api
- Auto-scaling: Scale to 3 instances under high load
- Deployment: Triggered automatically via Azure Pipelines on main branch push
- Managed SSL/TLS certificate — HTTPS enforced
- Environment variables (DB_HOST, JWT_SECRET) stored in App Service Configuration

3. IaaS — Infrastructure as a Service

IaaS provides virtualized computing resources over the internet. Users manage the operating system, runtime, middleware, and applications, while the provider manages hardware, networking, and physical data centers.

Key Characteristics

- Full control over operating system and runtime environment
- Pay-as-you-go based on compute and storage consumption
- User responsible for OS patching, security, backups
- High flexibility — install any software stack
- Examples: Azure Virtual Machines, AWS EC2, Google Compute Engine

CrimeTrack — IaaS Usage: Azure Virtual Machine (MySQL)

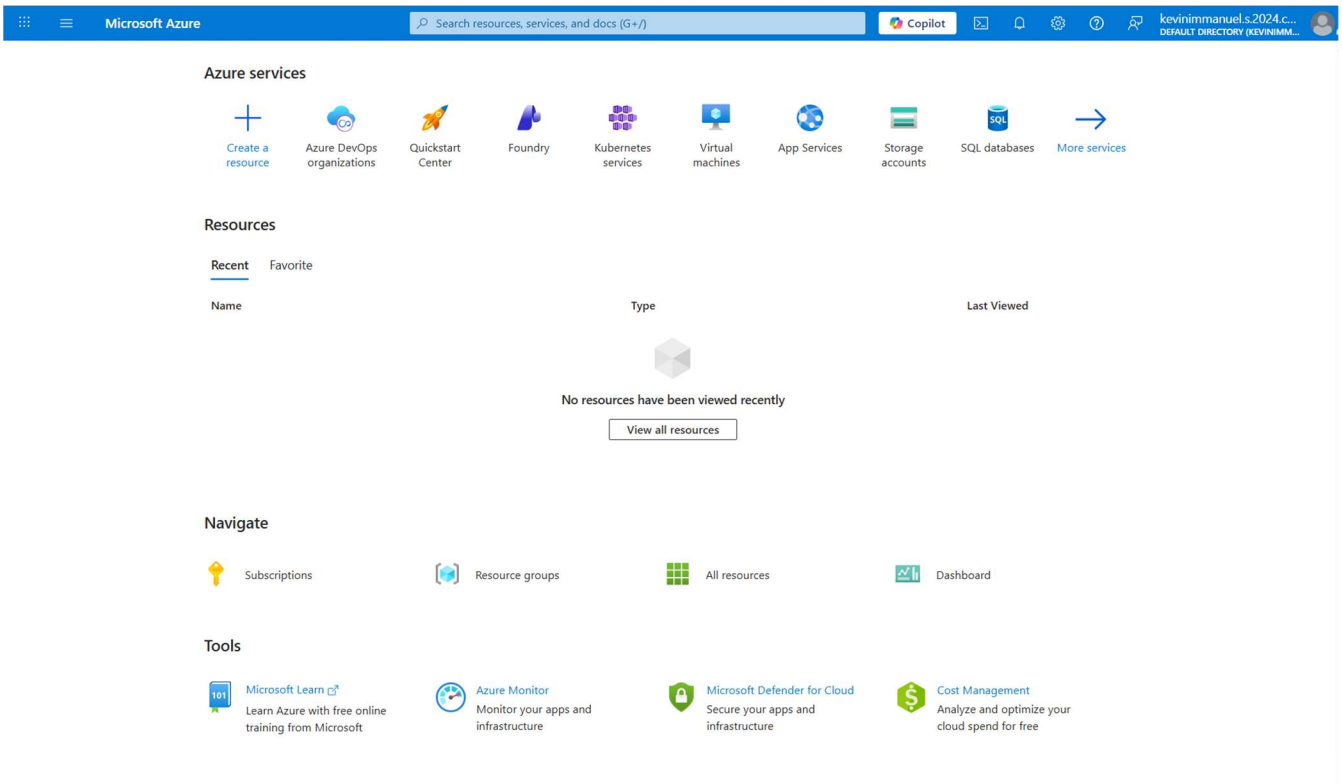
The CrimeTrack MySQL 8.0 database server is hosted on an Azure Virtual Machine (IaaS):

- VM: Standard_B2s (2 vCPUs, 4 GB RAM) — Ubuntu 22.04 LTS
- MySQL 8.0 installed and configured manually on the VM
- Database: crimetrack_db with tables: users, complaints

- Firewall rules: Port 3306 accessible only from App Service outbound IP
- Automated daily backups via Azure Backup Vault
- Static Private IP assigned — not exposed to public internet

Comparison: SaaS vs PaaS vs IaaS for CrimeTrack

Aspect	SaaS (Azure DevOps)	PaaS (App Service)	IaaS (VM — MySQL)
Control Level	Minimal	Medium	Full
Management Effort	None	Low	High
Used For	Project Management	Backend API Hosting	Database Server
Scaling	Automatic	Automatic	Manual
Cost Model	Per user/month	Per app/hour	Per VM/hour
Security Mgmt	Provider-managed	Shared	User-managed



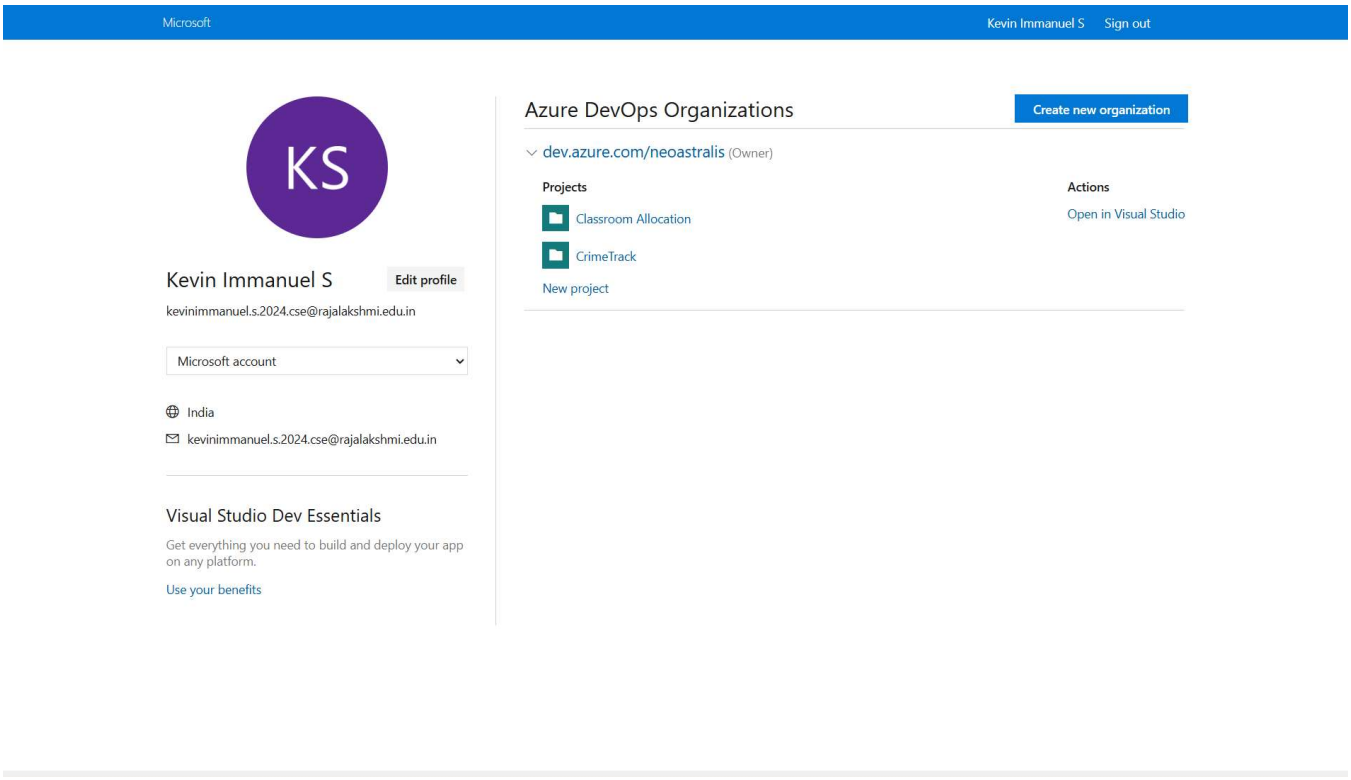
AZURE DEVOPS

Azure DevOps is a cloud-based platform by Microsoft that provides tools for DevOps practices, including CI/CD pipelines, version control, agile planning, testing, and monitoring. It supports teams in automating software development and deployment.

Steps to Create Azure DevOps Organization

1. **Open Azure DevOps Portal**
   Navigate to the Azure DevOps website using a web browser.
2. **Sign in to Your Account**
   Log in using your Microsoft account credentials.
3. **Create a New Organization**
   - Click on **"New Organization"**
   - Enter a unique **Organization Name**
   - Select the appropriate **Region**
   - Accept the terms and conditions
     - Click **Create**

4. **Create a New Project**
   - Enter the **Project Name** (e.g., CrimeTrack)
   - Set **Visibility** (Public/Private)
   - Select **Version Control System** (Git recommended)
5. **Select Process as Agile**
   - In the process option, choose **"Agile"**
   - This enables features like user stories, sprints, backlog, and task tracking
6. **Create the Project**
   Click on **"Create Project"** to complete the setup.
7. **Access Agile Tools**
   After creation, use:
   - **Boards** → Manage user stories, tasks, and sprints
   - **Backlogs** → Plan and prioritize work
   - **Sprints** → Track progress in iterations



Result:

Thus, Microsoft Azure and its services were successfully studied, including cloud models, features, and web application services. An Azure account and Azure DevOps organization with Agile process were created, studied successfully.

Experiment - 2

Problem Statement

Traditional crime reporting systems are mostly manual and time-consuming. Citizens must visit police stations physically to file complaints, which leads to:

- Delay in complaint registration
- Lack of transparency in tracking case status
- Poor communication between citizens and authorities
- Risk of data loss in manual records

Objectives

- Provide an online platform for citizens to report crimes easily
- Enable real-time complaint tracking
- Improve communication between citizens and admin (police)
- Maintain secure and centralized data storage
- Reduce manual workload and increase efficiency

Proposed Solution

- Develop a web-based application (CrimeTrack / CitizenTrack) with:
- User registration and login (Citizen & Admin)
- Online complaint submission with details and location
- Evidence upload (images/documents)
- Unique complaint ID generation
- Admin dashboard to manage and update complaints
- Real-time status tracking for users
- Notification system for updates

Key Functionalities

The CrimeTrack system provides a digital platform that allows citizens to easily report and track crime-related complaints without visiting police stations. It focuses on improving accessibility, transparency, and efficiency in the complaint handling process. Users can securely register, log in, and submit complaints with detailed information, including location and supporting evidence.

From the administrative side, the system enables authorities to manage complaints effectively through a centralized dashboard. Admins can view, filter, and update complaint statuses while maintaining communication with users through notifications. The system also ensures data security and proper access control, making it reliable and scalable.

- User registration and secure login (Citizen & Admin)
- Online complaint submission with details and category
- Location selection using map integration
- Upload evidence (images/documents)
- Unique complaint ID generation
- Real-time complaint status tracking
- Admin dashboard for managing complaints
- Status update and remarks by admin
- Notification system for user updates
- Secure data storage and role-based access control

Experiment-3

1.1 Functional Requirements — User Stories

Functional requirements define what the system must do. In Azure DevOps, these are captured as User Stories on the Backlog Board under Epic: CrimeTrack Portal Development.

Epic Structure in Azure DevOps

```
Epic: CrimeTrack Portal Development
  Feature: User Authentication System
    User Story: US-001 — Citizen Registration
    User Story: US-002 — Citizen Login with JWT
  Feature: Complaint Management
    User Story: US-003 — File Crime Complaint
    User Story: US-004 — Track Complaint by ID
  Feature: Admin Dashboard
    User Story: US-005 — View and Filter All Complaints
    User Story: US-006 — Update Complaint Status
```

User Story Format (Azure DevOps Template)

All user stories follow the standard format: "As a [role], I want to [action], so that [benefit]."

User Stories Table

Story ID	User Story	Role	Priority	Sprint
US-001	As a citizen, I want to register with email & password so that I can access the portal	Citizen	Must Have	Sprint 1
US-002	As a citizen, I want to log in securely so that my data is protected	Citizen	Must Have	Sprint 1
US-003	As a citizen, I want to file a crime complaint with details so that authorities can act	Citizen	Must Have	Sprint 1
US-004	As a citizen, I want to track my complaint using a unique ID so that I know its status	Citizen	Must Have	Sprint 1
US-005	As an admin, I want to view all complaints so that I can prioritize cases	Admin	Must Have	Sprint 2

Story ID	User Story	Role	Priority	Sprint
US-006	As an admin, I want to update complaint status so that citizens are informed	Admin	Must Have	Sprint 2
US-007	As a citizen, I want to receive status badges on complaints so that I track progress visually	Citizen	Should Have	Sprint 2
US-008	As an admin, I want to search complaints by type/location so that I can filter relevant cases	Admin	Should Have	Sprint 2

Homepage User Story — US-003: File Crime Complaint (Detailed)

This is the primary user story of the CrimeTrack portal and demonstrates the homepage/core functionality.

> User Story ID: US-003
> Title: File a Crime Complaint
>
> As a registered citizen,
> I want to fill out a complaint form with crime type, description, date, and location,
> So that I can formally report an incident to the relevant authorities and receive a
> unique complaint tracking ID for future follow-up.
>
> Priority: Must Have | Story Points: 8 | Sprint: Sprint 1
> Assigned To: Developer Team | Status: Done

Acceptance Criteria (Definition of Done — Azure DevOps)

- Citizen must be logged in to access the complaint form
- Form must include: Crime Type (dropdown), Description (textarea, min 20 chars), Date (date picker, not future), Location (text)
- On submit, system generates a unique Complaint ID (format: CT{YEAR}{5-digit random})
- Complaint is persisted in MySQL database with status = 'pending'
- Success screen shows Complaint ID with Copy to Clipboard button
- Form validates all fields before submission — inline error messages displayed
- Unauthorized access (no token) redirects to login page
- Unit tests pass for complaint ID generation and validation logic

1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system — how well it performs, scales, and secures data.

ID	Category	Requirement	Metric / Target
NFR-001	Performance	Page load time must be acceptable	< 2 seconds on 4G network
NFR-002	Scalability	System must handle concurrent users	500 simultaneous users without degradation
NFR-003	Security	Passwords must be hashed securely	bcrypt with 12 salt rounds
NFR-004	Security	All API endpoints secured with JWT	Token expiry: 7 days; HTTPS enforced
NFR-005	Availability	Portal must be highly available	99.9% uptime SLA via Azure App Service
NFR-006	Usability	Responsive design across devices	Mobile, tablet, and desktop — Tailwind CSS
NFR-007	Maintainability	Code must be modular and documented	Components separated; API routes documented
NFR-008	Data Integrity	Database must use parameterised queries	Prevent SQL injection; mysql2 library used
NFR-009	Compliance	Government data privacy	No PII exposed in API responses unnecessarily
NFR-010	Auditability	All complaint status changes logged	Updated_at timestamp stored in MySQL

1.3 Azure DevOps Backlog Board

Azure DevOps Boards provide a visual Kanban/Scrum management interface for CrimeTrack. The backlog is organized into Epics, Features, User Stories, Tasks, and Bugs.

Sprint 1 Board — Column Structure

Backlog	Active (To Do)	In Progress	Review	Done
US-007	US-001	US-003	US-002	Project Setup
US-008	US-004	US-005		DB Schema Design

Backlog	Active (To Do)	In Progress	Review	Done
				Azure DevOps Setup

Work Item Hierarchy in Azure DevOps

- Epic: CrimeTrack Portal Development (covers entire project)
- Feature: User Authentication System, Complaint Management, Admin Panel
- User Story: Individual functional requirements (US-001 to US-008)
- Task: Sub-tasks under each user story (e.g., 'Design Login UI', 'Write JWT middleware')
- Bug: Defects found during testing (tracked with severity and repro steps)

Each work item in Azure DevOps includes: Title, Description, Acceptance Criteria, Story Points, Priority, Sprint Assignment, Assigned To, and State.

1.4 Planning Poker — Story Points and Justification

Planning Poker is an agile estimation technique where team members independently assign story points to user stories using Fibonacci cards (1, 2, 3, 5, 8, 13, 20) to avoid anchoring bias. Story points measure complexity, effort, and uncertainty — not time.

How Planning Poker Works in Azure DevOps

- Team meets in Sprint Planning session — stories from backlog are reviewed
- Each team member independently selects a card (1, 2, 3, 5, 8, 13, 20)
- All cards revealed simultaneously — avoids anchoring bias
- If estimates diverge, the highest and lowest estimators explain rationale
- Discussion continues until consensus is reached
- Final points entered in Azure DevOps User Story 'Story Points' field

CrimeTrack — Planning Poker Results

Title	Dev A	Dev B	Dev C	Final Points	Justification
Citizen Registration	5	5	3	5	Form + bcrypt + DB insert; moderate complexity
Citizen Login (JWT)	3	5	3	3	Standard JWT flow; less complex than registration

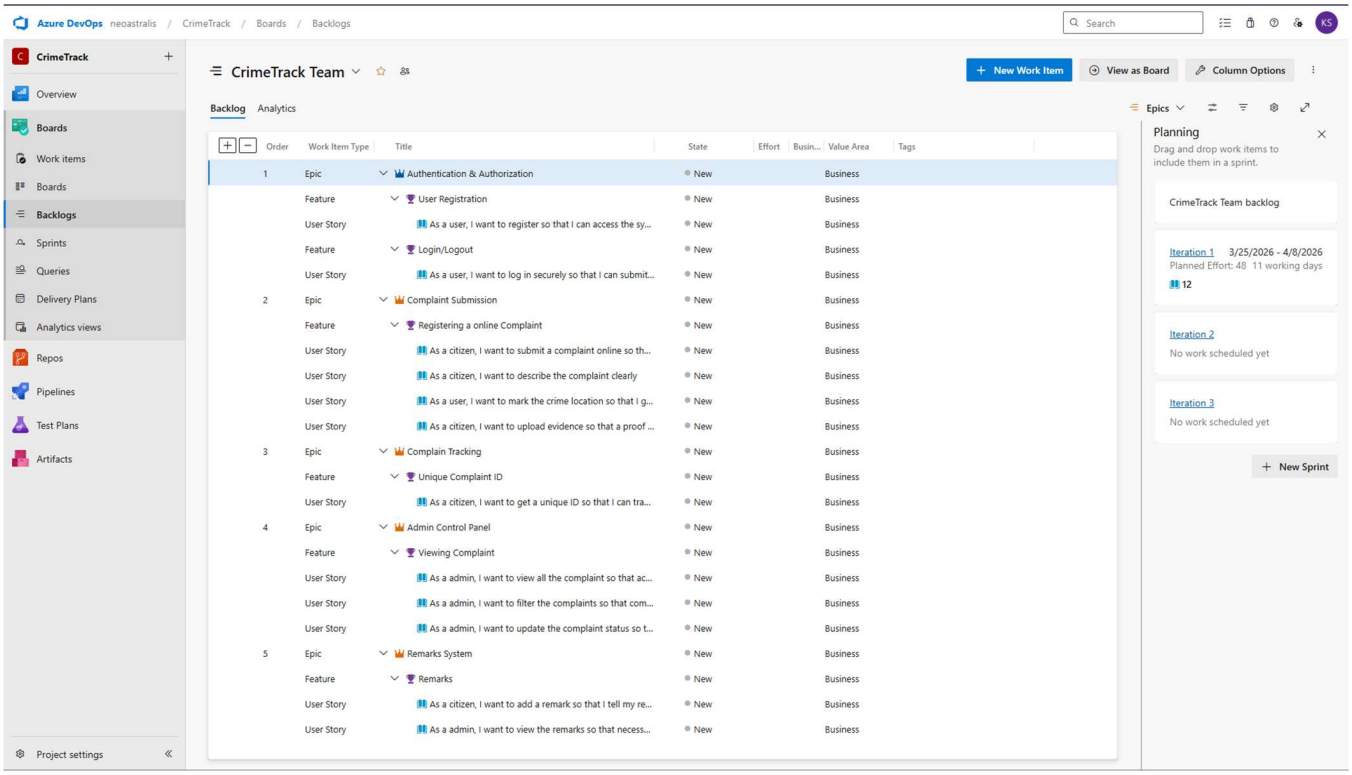
Title	Dev A	Dev B	Dev C	Final Points	Justification
File Complaint Form	8	8	13	8	Multi-field form, validation, ID gen, DB — high effort
Track Complaint by ID	3	3	5	3	Single query + status display; low complexity
Admin View All Complaints	8	5	8	8	Pagination, filters, search — significant backend work
Update Complaint Status	5	5	5	5	Dropdown + PATCH API + optimistic UI update
Status Badges on Complaints	2	2	3	2	Pure UI component — minimal logic
Admin Search & Filter	5	8	5	5	Query builder with multiple params — moderate effort

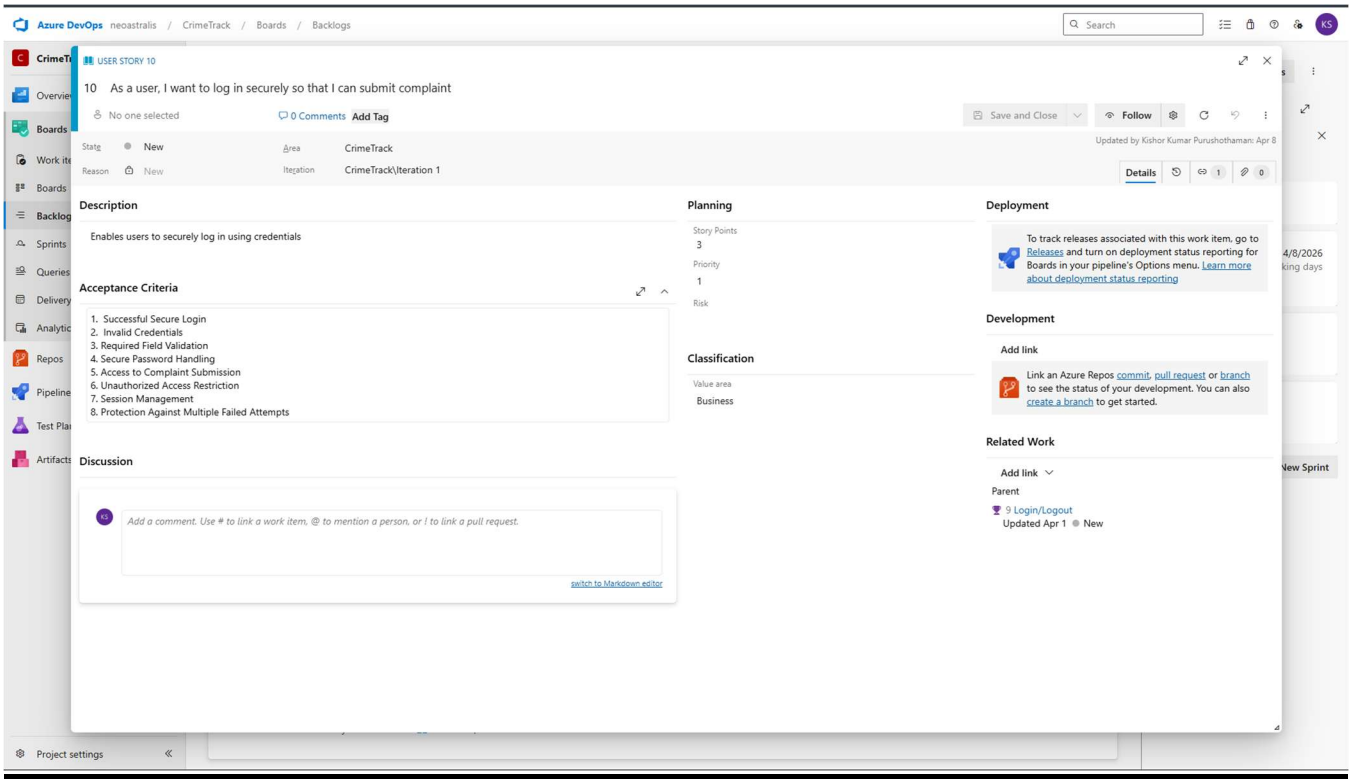
Story Point Scale Reference

Points	Complexity Level	Typical Scope
1	Trivial	Text change, config update
2	Simple	Basic UI component, read-only API
3	Low	CRUD operation with basic validation
5	Medium	Feature with multiple components + API
8	High	Complex feature, multiple integrations
13	Very High	Architectural change, significant risk
20	Epic-level	Break into smaller stories

Total Sprint Velocity

Sprint 1 Total Story Points: 19
Sprint 2 Total Story Points: 20
Team Velocity (avg): 19-20 points/sprint
Sprint Duration: 2 weeks | Team Size: 3 developers


Azure DevOps neoastralis / CrimeTrack / Boards / Backlogs
Search
CrimeTrack
Overview
Boards
Work items
Boards
Backlogs
Sprints
Queries
Delivery Plans
Analytics views
Repos
Pipelines
Test Plans
Artifacts
Project settings
CrimeTrack Team
New Work Item
View as Board
Column Options
Backlog
Analytics
Epics
Order
Work Item Type
Title
State
Effort
Busin...
Value Area
Tags
1 Epic Authentication & Authorization New Business
Feature User Registration New Business
User Story As a user, I want to register so that I can access the sy... New Business
Feature Login/Logout New Business
User Story As a user, I want to log in securely so that I can submit... New Business
2 Epic Complaint Submission New Business
Feature Registering a online Complaint New Business
User Story As a citizen, I want to submit a complaint online so th... New Business
User Story As a citizen, I want to describe the complaint clearly New Business
User Story As a user, I want to mark the crime location so that I g... New Business
User Story As a citizen, I want to upload evidence so that a proof ... New Business
3 Epic Complain Tracking New Business
Feature Unique Complaint ID New Business
User Story As a citizen, I want to get a unique ID so that I can tra... New Business
4 Epic Admin Control Panel New Business
Feature Viewing Complaint New Business
User Story As a admin, I want to view all the complaint so that ac... New Business
User Story As a admin, I want to filter the complaints so that com... New Business
User Story As a admin, I want to update the complaint status so t... New Business
5 Epic Remarks System New Business
Feature Remarks New Business
User Story As a citizen, I want to add a remark so that I tell my re... New Business
User Story As a admin, I want to view the remarks so that necess... New Business
Planning
Drag and drop work items to include them in a sprint.
CrimeTrack Team backlog
Iteration 1 3/25/2026 - 4/8/2026
Planned Effort: 48 11 working days
12
Iteration 2
No work scheduled yet
Iteration 3
No work scheduled yet
New Sprint


Azure DevOps neoastralis / CrimeTrack / Boards / Backlogs
Search
USER STORY 10
10 As a user, I want to log in securely so that I can submit complaint
No one selected
0 Comments
Add Tag
Save and Close
Follow
Updated by Kishor Kumar Purushothaman: Apr 8
State New
Reason New
Area CrimeTrack
Iteration CrimeTrack\Iteration 1
Details
Description
Enables users to securely log in using credentials
Acceptance Criteria
1. Successful Secure Login
2. Invalid Credentials
3. Required Field Validation
4. Secure Password Handling
5. Access to Complaint Submission
6. Unauthorized Access Restriction
7. Session Management
8. Protection Against Multiple Failed Attempts
Discussion
Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.
switch to Markdown editor
Planning
Story Points
3
Priority
1
Risk
Classification
Value area
Business
Deployment
To track releases associated with this work item, go to Releases and turn on deployment status reporting for Boards in your pipeline's Options menu. Learn more about deployment status reporting
Development
Add link
Link an Azure Repos commit, pull request or branch to see the status of your development. You can also create a branch to get started.
Related Work
Add link
Parent
9 Login/Logout
Updated Apr 1 New

Experiment - 4

Use Case Diagram

What is a Use Case Diagram?

A Use Case Diagram is a UML (Unified Modeling Language) behavioral diagram that describes the functional interactions between users (actors) and a system. It shows what the system does (use cases) from the user's perspective, without describing how it is done internally.

Use case diagrams are the first step in requirements engineering — created during the Problem Understanding phase and linked to User Stories in Azure DevOps. Each use case corresponds to one or more user stories on the backlog.

Notations and Symbols

Symbol	Name	Description
Oval / Ellipse	Use Case	Represents a specific function or action the system performs (e.g., File Complaint)
Stick Figure	Actor	External entity (user or system) interacting with the system (e.g., Citizen, Admin)
Rectangle	System Boundary	Defines the scope of the system — use cases inside, actors outside
Solid Arrow	Association	Connects an actor to a use case they participate in
<<include>> Arrow	Include Relationship	A use case always invokes another (mandatory dependency)
<<extend>> Arrow	Extend Relationship	A use case optionally adds behaviour to another (conditional)
Hollow Arrow (Inheritance)	Generalization	One actor/use case inherits from another (IS-A relationship)

CrimeTrack — Use Case Diagram Description

The CrimeTrack system has two primary actors: Citizen and Admin. The system boundary is the CrimeTrack Web Portal, which is deployed on Azure App Service.

Actors

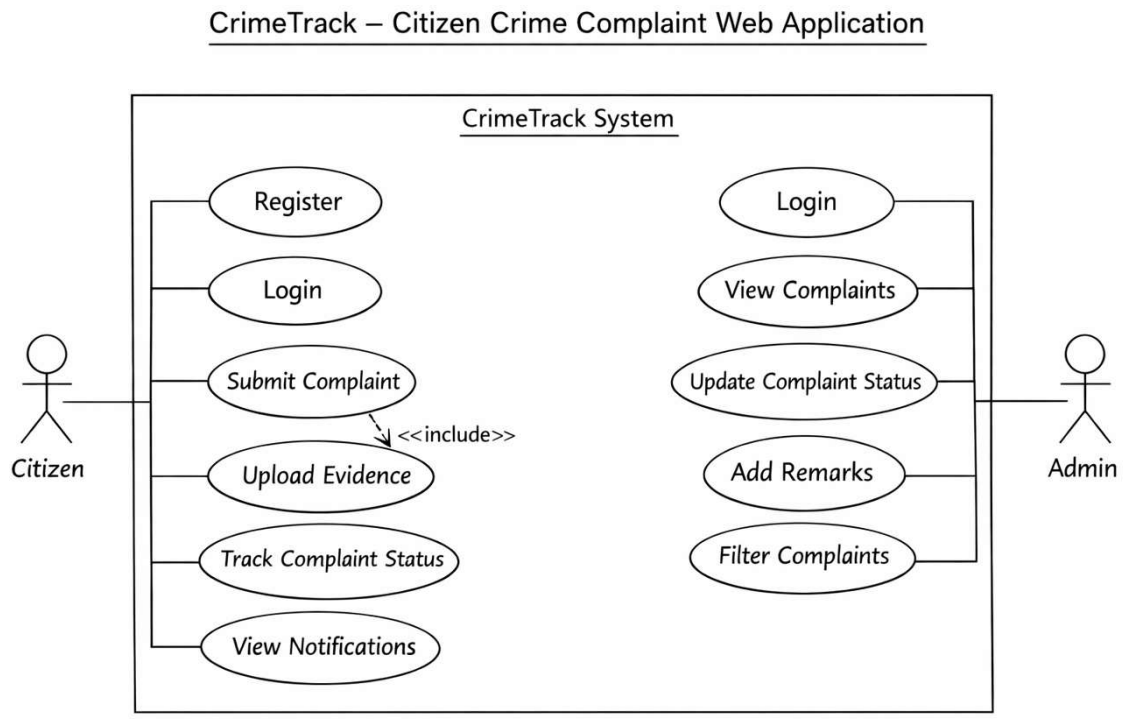
- Citizen — Registered user who files and tracks complaints
- Admin — Government officer who manages and resolves complaints
- Azure DevOps Pipeline (System Actor) — Automates deployment on code push

Use Cases and Relationships

Use Case	Actor(s)	Relationship	Azure DevOps Story
Register Account	Citizen	Primary	US-001
Login / Authenticate	Citizen, Admin	Primary	US-002
Validate JWT Token	System	<<include>> from Login	US-002
File Crime Complaint	Citizen	Primary	US-003
Generate Complaint ID	System	<<include>> from File Complaint	US-003
View Complaint History	Citizen	Primary	US-003
Track Complaint by ID	Citizen, Public	Primary	US-004
Display Status Timeline	System	<<extend>> from Track Complaint	US-004
View All Complaints	Admin	Primary	US-005
Filter and Search Complaints	Admin	<<extend>> from View All	US-008
Update Complaint Status	Admin	Primary	US-006
Add Admin Notes	Admin	<<extend>> from Update Status	US-006
View Dashboard Statistics	Admin	Primary	US-005
Logout	Citizen, Admin	Primary	-

How to Draw a Use Case Diagram

- Step 1: Identify all actors (humans or external systems) — Citizen, Admin
- Step 2: Identify all use cases from user stories — one story = one or more use cases
- Step 3: Draw the system boundary rectangle — label it 'CrimeTrack Portal'
- Step 4: Place actors as stick figures outside the boundary
- Step 5: Place use cases as ovals inside the boundary
- Step 6: Draw association lines between actors and their use cases
- Step 7: Add <<include>> arrows for mandatory sub-use-cases
- Step 8: Add <<extend>> arrows for optional/conditional behaviour
- Step 9: Add generalization if Admin IS-A type of User (inherits Login)



Experiment - 5

Class Diagram

What is a Class Diagram?

A Class Diagram is a UML structural diagram that describes the static structure of a system by showing classes, their attributes, methods, and the relationships between them. It represents the blueprint of the application's object-oriented design and directly maps to the database schema and backend models in CrimeTrack.

In Azure DevOps, the class diagram is created during the Design phase and linked to the relevant Feature/User Story in the Wiki. It guides developers in creating the MySQL schema and Express.js model layer.

Notations and Symbols

Symbol / Notation	Description	CrimeTrack Example
Rectangle (3 sections)	Represents a Class: top=name, middle=attributes, bottom=methods	User, Complaint classes
- (minus sign)	Private attribute or method — accessible only within the class	- password: String
+ (plus sign)	Public attribute or method — accessible from outside the class	+ getName(): String
# (hash)	Protected attribute — accessible by subclasses	# id: Integer
1	Multiplicity — exactly one instance	One User
0..* (zero to many)	Multiplicity — zero or more instances	A user has 0..* complaints
1..* (one to many)	Multiplicity — at least one instance	An order has 1..* items
——— (solid line)	Association — objects are aware of each other	User associates with Complaint
◆——— (filled diamond)	Composition — strong ownership, child cannot exist without parent	Complaint belongs to User
◇——— (empty diamond)	Aggregation — weak ownership, child can exist independently	Admin manages Complaints
——▷ (open arrow)	Inheritance / Generalization — IS-A relationship	Admin IS-A User
- - -▷ (dashed arrow)	Dependency / Realization	Controller uses Service

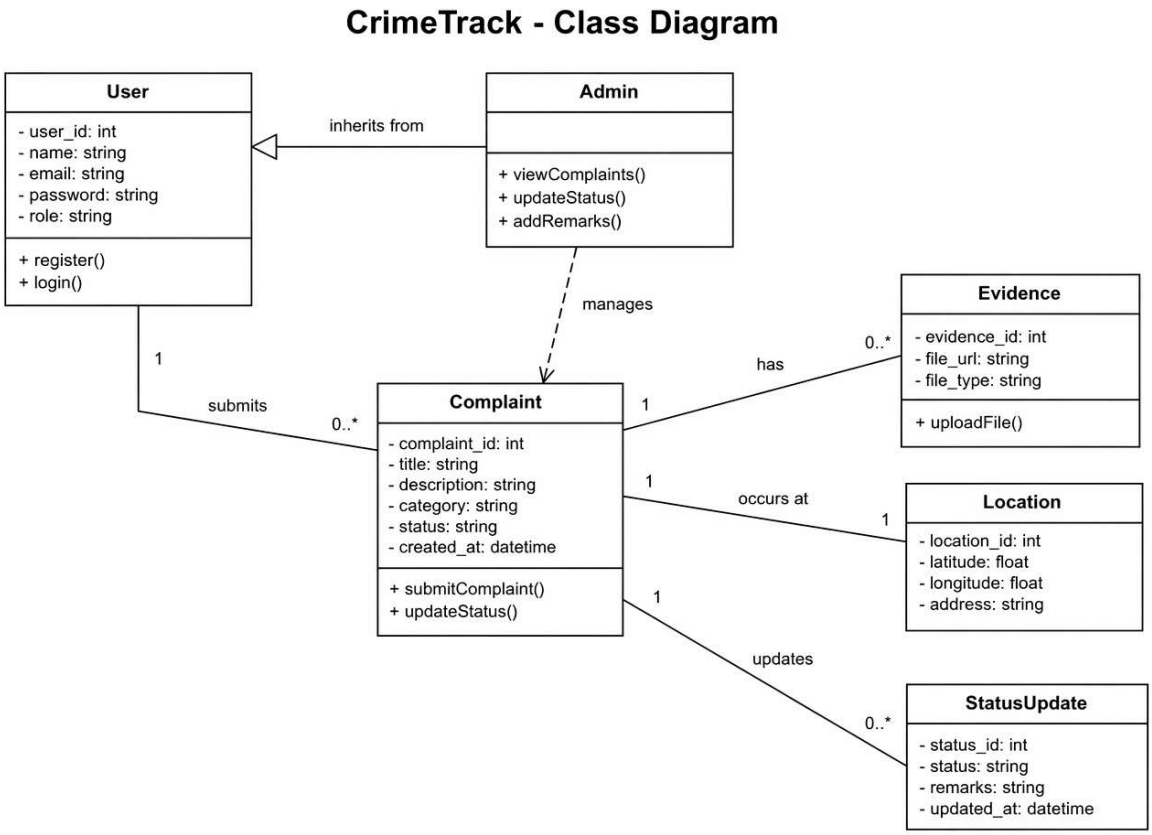
CrimeTrack — Class Diagram

Class Relationships

From Class	Relationship	To Class	Multiplicity	Description
User	Has (Composition)	Complaint	1 to 0..*	One user can file zero or many complaints
Admin	Generalization (extends)	User	IS-A	Admin is a specialization of User with extra permissions
AuthController	Dependency	User	uses	AuthController uses User class to register/login
ComplaintController	Dependency	Complaint	uses	ComplaintController delegates to Complaint model
AdminController	Dependency	Complaint	uses	AdminController manages Complaint status updates
Complaint	Association	User	Many to 1	Each complaint belongs to exactly one User

How to Draw a Class Diagram

- Step 1: Identify all entities/models from the database schema — User, Complaint
- Step 2: For each class, list attributes with data types and access modifiers
- Step 3: For each class, list methods/operations with return types
- Step 4: Identify relationships between classes — Association, Composition, Inheritance
- Step 5: Add multiplicity labels (1, 0..*, 1..*)
- Step 6: Draw inheritance hierarchies — Admin extends User
- Step 7: Add Controller classes that depend on model classes
- Step 8: Review against Azure DevOps user stories to ensure all features are modelled



Experiment - 6

Sequence Diagram

What is a Sequence Diagram?

A Sequence Diagram is a UML interaction diagram that shows how objects or components communicate with each other over time in a specific scenario. It captures the chronological order of messages exchanged between participants (actors, services, databases) to complete a use case.

For CrimeTrack, sequence diagrams document the exact flow of API calls, database queries, and UI state changes for each user story. These are referenced in Azure DevOps Sprint Planning and stored in the project Wiki.

Notations and Symbols

Symbol	Name	Description
Rectangle at top	Participant / Lifeline	Represents an object, class, or system component (e.g., Browser, API, DB)
Vertical dashed line	Lifeline	Represents the lifetime of an object during the interaction
Narrow rectangle on lifeline	Activation Box	Shows when an object is active/processing a request
Solid arrow →	Synchronous Message	A call that waits for a response (e.g., HTTP POST request)
Dashed arrow - - →	Return Message	Response to a previous synchronous call (e.g., 200 OK)
Open arrow →	Asynchronous Message	A call that does not wait for a response (e.g., event/webhook)
[condition] guard	Guard Condition	Message only sent if condition is true (e.g., [token valid])
loop / alt / opt frames	Combined Fragment	Loop, alternative flows, and optional message sequences
X at end of lifeline	Destruction	Object is destroyed/session ends

CrimeTrack — Sequence Diagram: File Complaint (US-003)

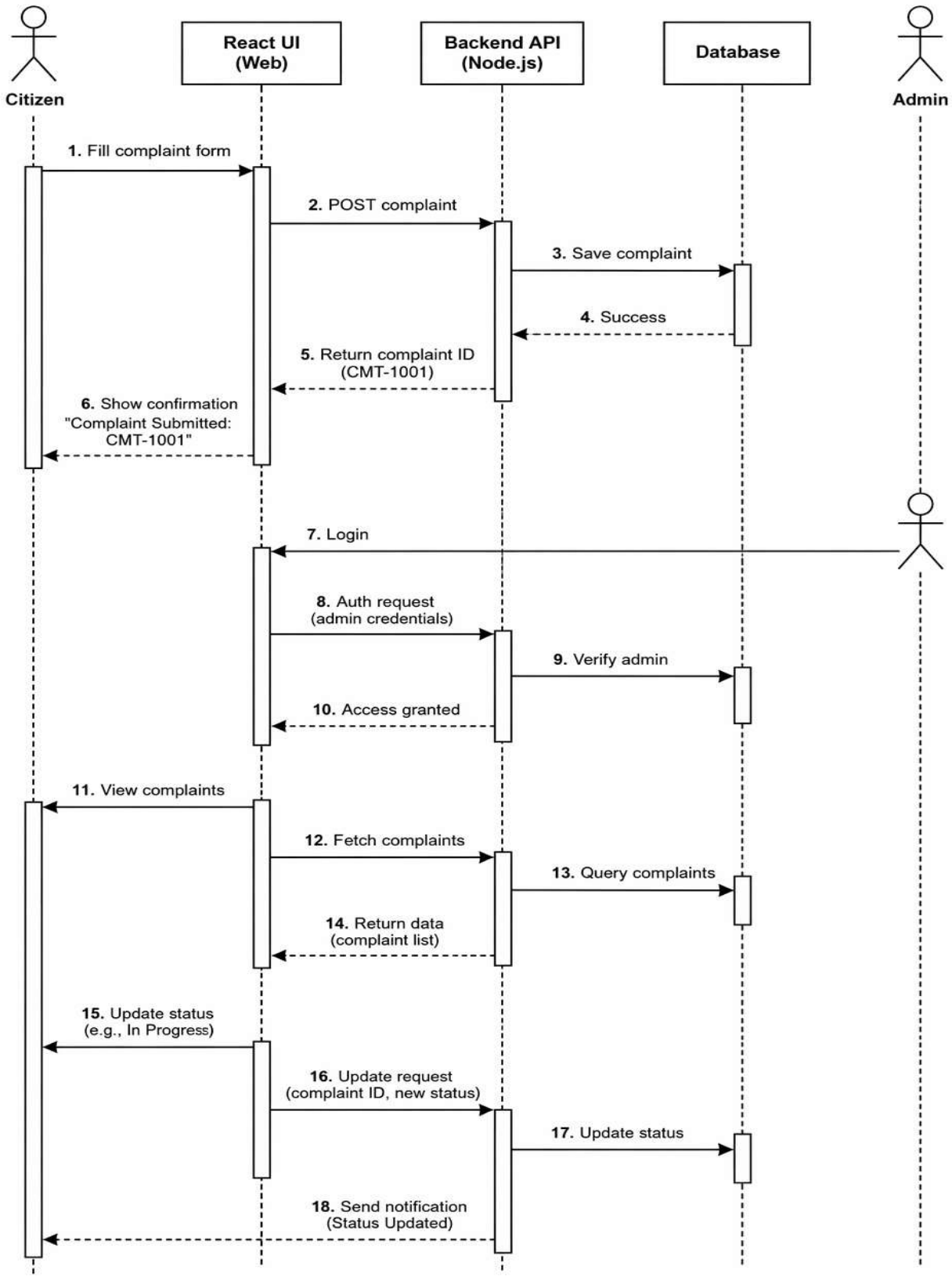
This sequence diagram captures the complete flow when a citizen files a crime complaint, corresponding to the primary user story US-003.

Participants

- Citizen Browser (React Frontend)
- React Component (FileComplaint.jsx)
- Axios API Client (api.js)
- Express Server (complaints.js route)
- JWT Middleware (auth.js)
- MySQL Database (crimetrack_db)

How to Draw a Sequence Diagram

- Step 1: Identify the use case / user story being modelled (e.g., File Complaint)
- Step 2: List all participants — Browser, Frontend, Backend, Middleware, Database
- Step 3: Draw participant boxes at the top; add lifelines (dashed vertical lines)
- Step 4: Number each message chronologically top-to-bottom
- Step 5: Draw solid arrows for requests; dashed arrows for responses
- Step 6: Add activation boxes to show processing duration
- Step 7: Add alt/opt fragments for conditional flows (e.g., [token invalid])
- Step 8: Add guard conditions in square brackets on arrows
- Step 9: Annotate messages with HTTP method and endpoint (e.g., POST /api/complaints)


Citizen
React UI (Web)
Backend API (Node.js)
Database
Admin
1. Fill complaint form
2. POST complaint
3. Save complaint
4. Success
5. Return complaint ID (CMT-1001)
6. Show confirmation "Complaint Submitted: CMT-1001"
7. Login
8. Auth request (admin credentials)
9. Verify admin
10. Access granted
11. View complaints
12. Fetch complaints
13. Query complaints
14. Return data (complaint list)
15. Update status (e.g., In Progress)
16. Update request (complaint ID, new status)
17. Update status
18. Send notification (Status Updated)

Experiment - 7

Entity-Relationship (ER) Diagram

What is an ER Diagram?

An Entity-Relationship (ER) Diagram is a data modelling diagram that visually represents the logical structure of a database. It shows entities (tables), their attributes (columns), and the relationships between them (foreign keys / associations).

The CrimeTrack ER diagram directly maps to the MySQL database schema created in db.js. The diagram is created during the Design phase and documented in Azure DevOps Wiki under 'Database Design'.

Notations and Symbols (Chen / Crow's Foot Notation)

Symbol	Name	Description
Rectangle	Entity	Represents a database table (e.g., users, complaints)
Oval	Attribute	Represents a column in the table (e.g., name, email)
Double Oval	Multi-valued Attribute	Attribute that can hold multiple values (e.g., tags)
Underlined Attribute	Primary Key (PK)	Uniquely identifies each row in the entity
Dashed Oval	Derived Attribute	Calculated from other attributes (e.g., age from DOB)
Diamond	Relationship	Describes how two entities are related (e.g., 'Files')
Double Diamond	Identifying Relationship	Child entity cannot exist without the parent entity
Single line |	One (1)	Crow's Foot: exactly one
Double line ||	Mandatory One	Crow's Foot: one and only one
Crow's foot ><	Many (*)	Crow's Foot: zero or more
Circle ○	Zero	Crow's Foot: zero instances allowed
─FK─	Foreign Key	Links child entity to parent entity's primary key

CrimeTrack — Database Entities

Entity 1: users

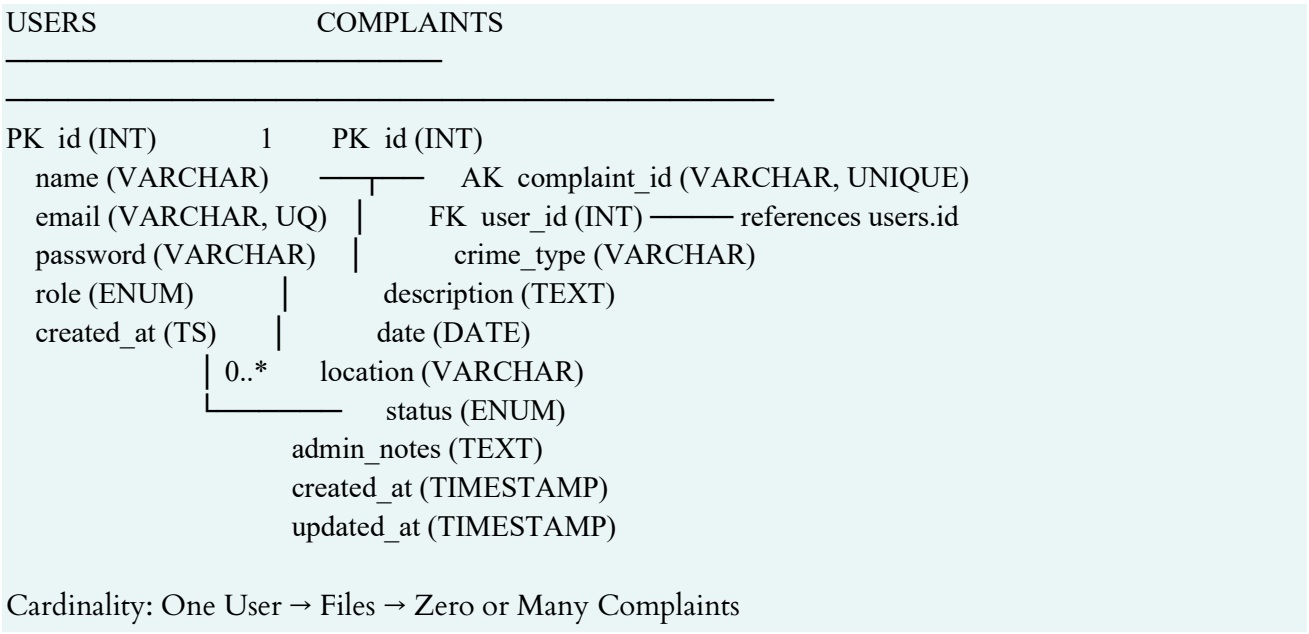
Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Unique user identifier
name	VARCHAR(255)	NOT NULL	Full name of the user
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email — must be unique
password	VARCHAR(255)	NOT NULL	bcrypt hashed password (12 rounds)
role	ENUM	DEFAULT 'citizen'	User role: 'citizen' or 'admin'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation time

Entity 2: complaints

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT, NOT NULL	Internal complaint row ID
complaint_id	VARCHAR(20)	UNIQUE, NOT NULL	Public tracking ID (CT2024XXXXX)
user_id	INT	FK -> users.id, NOT NULL	Links complaint to the filing citizen
crime_type	VARCHAR(100)	NOT NULL	Category of crime reported
description	TEXT	NOT NULL	Detailed description of the incident
date	DATE	NOT NULL	Date of incident (not future)
location	VARCHAR(255)	NOT NULL	Location of the incident
status	ENUM	DEFAULT 'pending'	pending / in_progress / resolved / rejected
admin_notes	TEXT	NULL	Notes added by admin during processing
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Complaint filing timestamp

Column	Data Type	Constraints	Description
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last status update timestamp

ER Diagram (Text Representation)

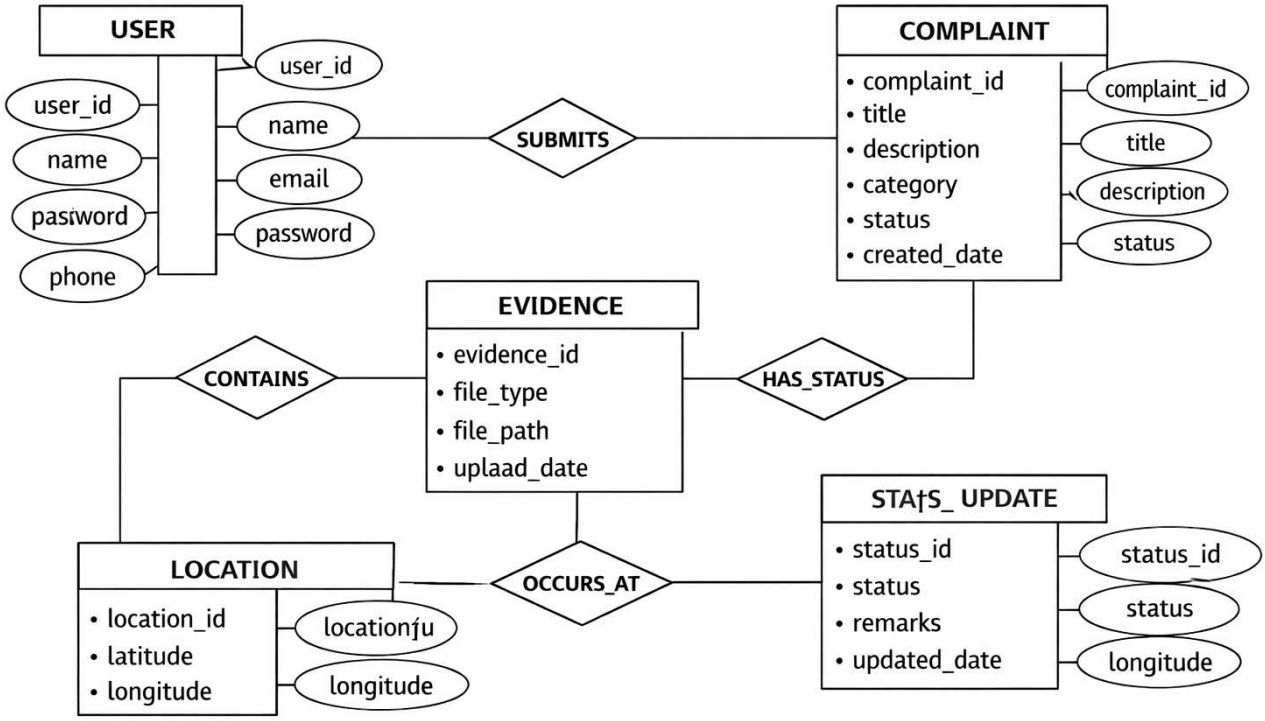


How to Draw an ER Diagram

- Step 1: Identify all entities from the database schema — users, complaints
- Step 2: List attributes for each entity; underline the primary key
- Step 3: Identify relationships between entities (Files, Manages)
- Step 4: Add cardinality labels using Crow's Foot or Chen notation
- Step 5: Mark foreign keys with dashed lines connecting to primary keys
- Step 6: Distinguish weak entities (Complaint cannot exist without User)
- Step 7: Use draw.io, Lucidchart, or Azure DevOps Wiki diagram to render

CrimeTrack

Citizen Crime Complaint Web Application



Experiment - 8

Activity Diagram

What is an Activity Diagram?

An Activity Diagram is a UML behavioral diagram that models the workflow or business process of a system. It shows the step-by-step sequence of activities, decision points, parallel flows, and termination conditions. It is particularly useful for visualizing use case flows, algorithm logic, and business processes.

In CrimeTrack, activity diagrams are used to document the workflow for filing complaints, login flow, and admin status updates. They are stored in the Azure DevOps Wiki as process documentation.

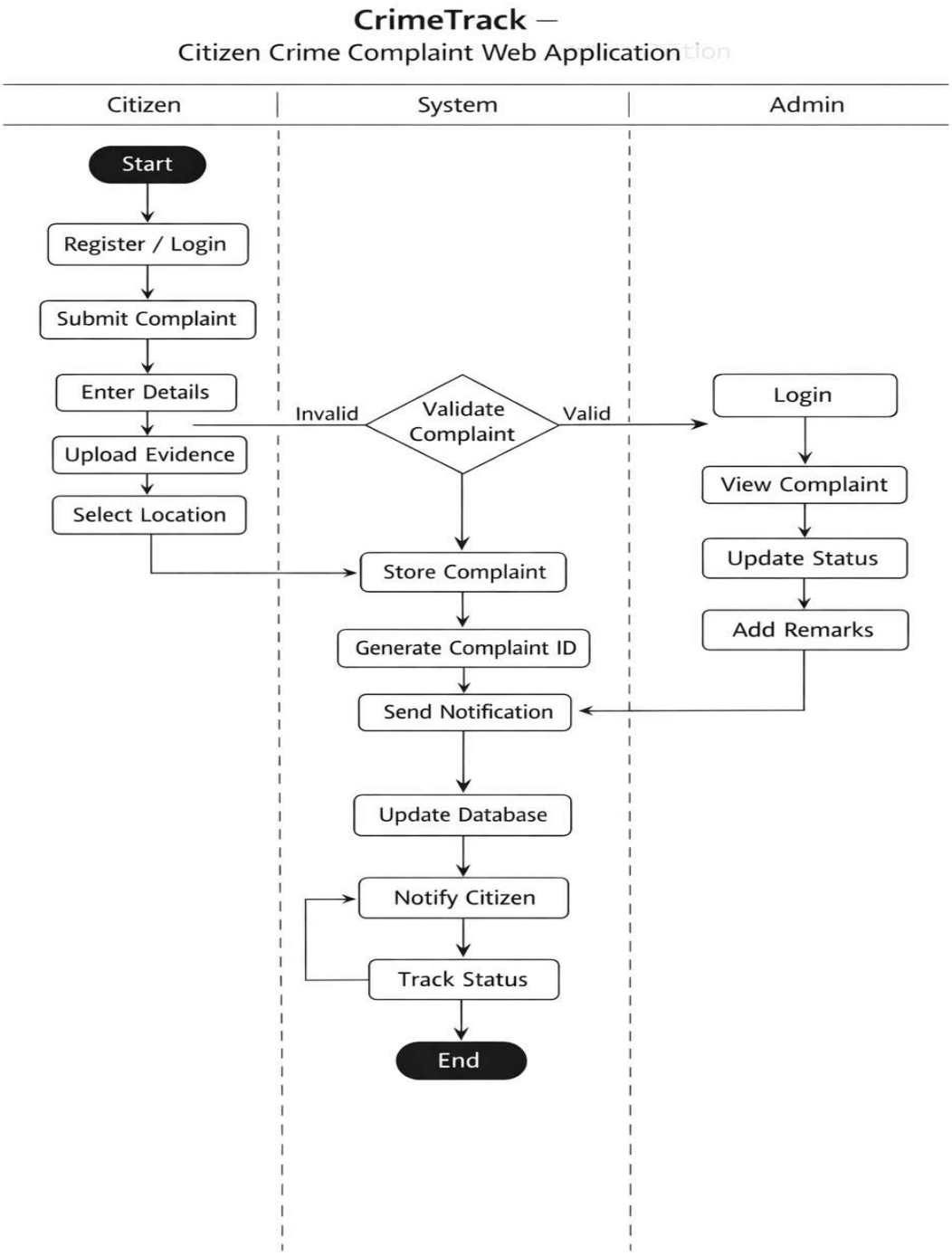
Notations and Symbols

Symbol	Name	Description
Filled Black Circle	Initial Node	Start point of the activity flow — there is exactly one
Filled Circle with Ring	Final Node (Activity End)	End point — terminates the entire activity
Filled Circle (flow end)	Flow Final Node	Terminates only one flow path, not the whole activity
Rounded Rectangle	Action / Activity	A single step or task in the workflow (e.g., 'Fill Form')
Diamond	Decision Node	A branching point with a condition — exactly one outgoing path taken
Diamond (join)	Merge Node	Combines multiple incoming paths back to one
Thick Horizontal Bar	Fork	Splits flow into multiple parallel flows (concurrent activities)
Thick Horizontal Bar	Join	Synchronizes multiple parallel flows back into one
Arrow	Control Flow	Shows direction of flow between activities
[condition]	Guard	Label on a flow arrow specifying when that path is taken
Swimlane / Partition	Swimlane	Groups activities by actor responsibility (e.g., Citizen, System, Admin)

How to Draw an Activity Diagram

- Step 1: Identify the process to model — a use case or business workflow
- Step 2: Define swimlanes based on actors — Citizen, System, Admin

- Step 3: Start with the Initial Node (filled black circle)
- Step 4: Add Activity nodes (rounded rectangles) for each step
- Step 5: Add Decision nodes (diamonds) for conditions and branches
- Step 6: Label all outgoing arrows from decision nodes with [guard conditions]
- Step 7: Add Fork/Join bars for any parallel activities
- Step 8: End with Activity Final Node (filled circle with ring)
- Step 9: Review against Azure DevOps acceptance criteria for completeness


CrimeTrack —
Citizen Crime Complaint Web Application
Citizen
System
Admin
Start
Register / Login
Submit Complaint
Enter Details
Upload Evidence
Select Location
Invalid
Validate Complaint
Valid
Login
View Complaint
Update Status
Add Remarks
Store Complaint
Generate Complaint ID
Send Notification
Update Database
Notify Citizen
Track Status
End

Experiment - 9

Architecture Diagram

What is an Architecture Diagram?

An Architecture Diagram is a high-level visual representation of the structural components, layers, and interactions of a software system. It shows how different services, servers, databases, and external integrations are connected and communicate with each other.

Architecture diagrams exist at multiple levels: System Architecture (high-level), Application Architecture (component-level), and Deployment Architecture (infrastructure-level). For CrimeTrack, all three views are relevant given the Azure cloud deployment.

Notations and Symbols

Symbol / Component	Description	CrimeTrack Representation
Rectangle / Box	Service, component, or server	React App, Express API, MySQL DB
Cylinder	Database	MySQL 8.0 on Azure VM
Cloud Shape	Cloud service or platform	Microsoft Azure Cloud
Browser Icon	Client / User Agent	Citizen or Admin Browser
Arrow →	Request/response flow	HTTP/HTTPS API calls
Dashed Arrow - →	Internal communication	API to DB connection
<<layer>> label	Architectural Layer	Presentation, Business Logic, Data Layer
Lock icon 🔒	Secured communication	HTTPS, JWT, bcrypt
Azure logo boxes	Azure-specific services	App Service, VM, DevOps, Backup
Numbered zones	Network boundaries	Public Internet, Azure VNET, Private Subnet

CrimeTrack — 3-Layer Application Architecture

Layer 1: Presentation Layer (Frontend)

- Technology: React 18 + Vite + Tailwind CSS + Framer Motion
- Hosted on: Azure Static Web Apps or served from Nginx
- Components: Navbar, Pages (Landing, Login, Register, Dashboard, FileComplaint, TrackComplaint, AdminDashboard)
- State Management: React Context API (AuthContext)

- HTTP Client: Axios with JWT interceptor

Layer 2: Business Logic Layer (Backend API)

- Technology: Node.js 18 + Express.js
- Hosted on: Azure App Service (PaaS — Linux, Node.js runtime)
- Routes: /api/auth, /api/complaints, /api/admin
- Middleware: JWT authentication, CORS, JSON body parser
- Libraries: bcryptjs (hashing), jsonwebtoken (JWT), uuid (ID generation), mysql2 (DB driver)

Layer 3: Data Layer (Database)

- Technology: MySQL 8.0
- Hosted on: Azure Virtual Machine (IaaS — Ubuntu 22.04 LTS)
- Tables: users, complaints
- Connection: mysql2 connection pool (max 10 connections)
- Security: Private IP, firewall rules — only App Service can connect on port 3306

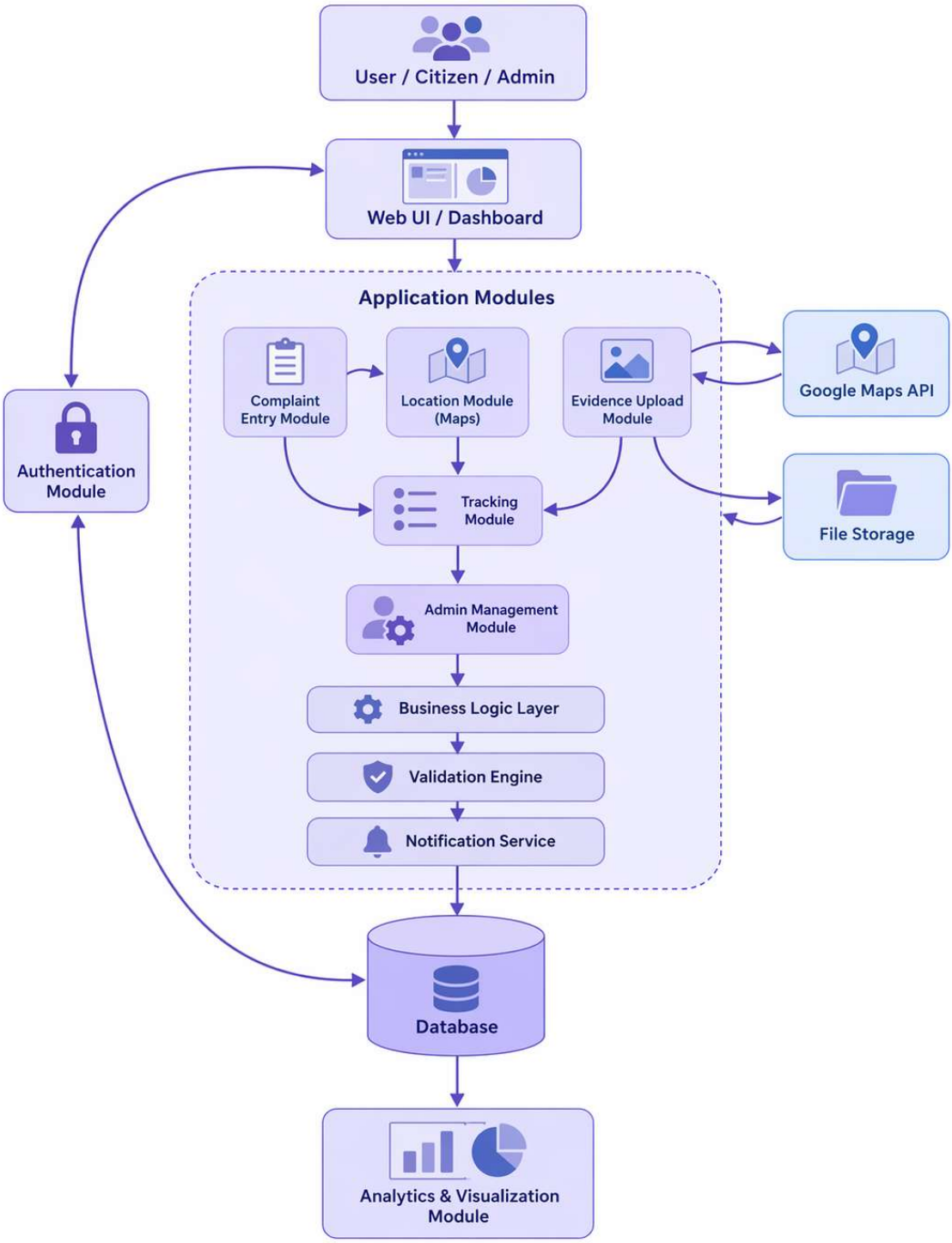
Architecture Diagram — Component Interaction Table

From	To	Protocol	Port	Security	Description
Browser (Citizen/Admin)	React Frontend	HTTPS	443	TLS 1.3	User accesses the web portal
React Frontend	Express API	HTTPS/REST	443	JWT Bearer Token	API calls for auth and complaints
Express API	MySQL Database	TCP/MySQL	3306	Private IP + Firewall	Database queries
Azure Pipelines	App Service	HTTPS/Kudu	443	Service Principal	CI/CD deployment
Azure Pipelines	Static Web Apps	HTTPS	443	API Token	Frontend deployment
Developer	Azure Repos	HTTPS/Git	443	SSH Key / PAT	Code push and pull requests

How to Draw an Architecture Diagram

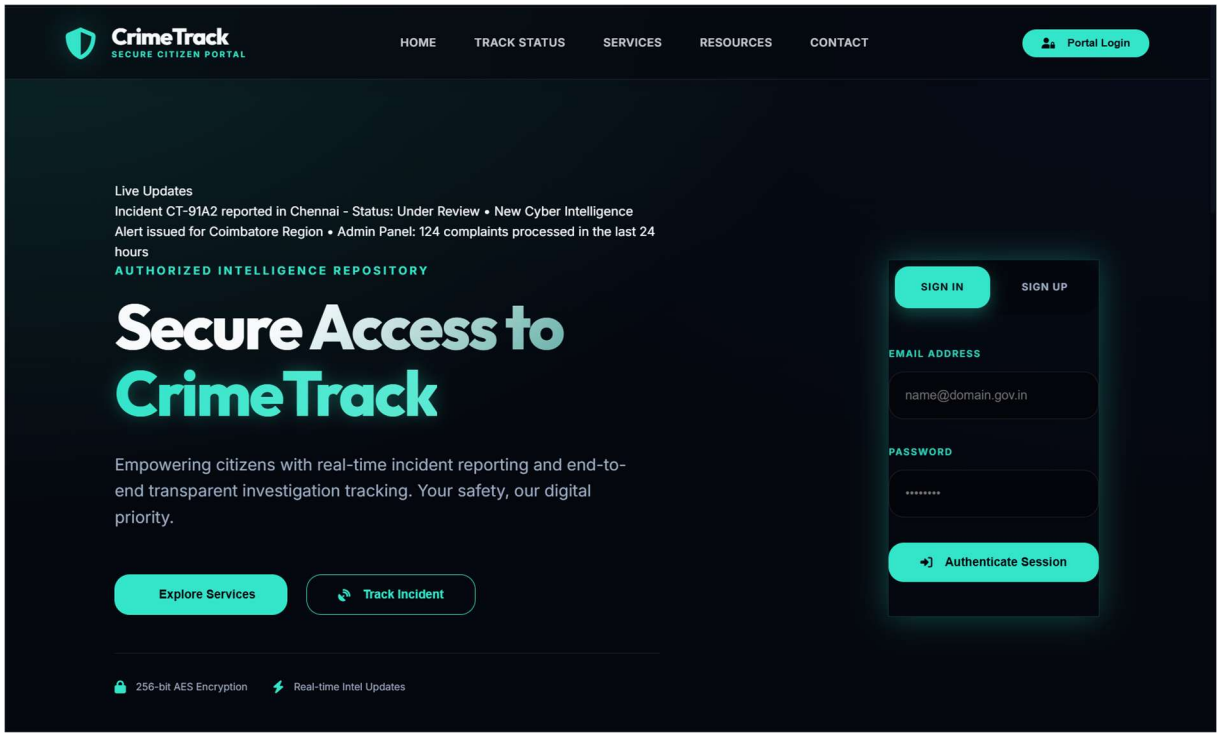
- Step 1: Identify all components — Frontend, Backend API, Database, CI/CD, Cloud services
- Step 2: Choose architecture style — Layered (3-tier), Microservices, or Event-driven
- Step 3: Draw client layer (Browser) at the top
- Step 4: Draw presentation layer (React Static Web App)

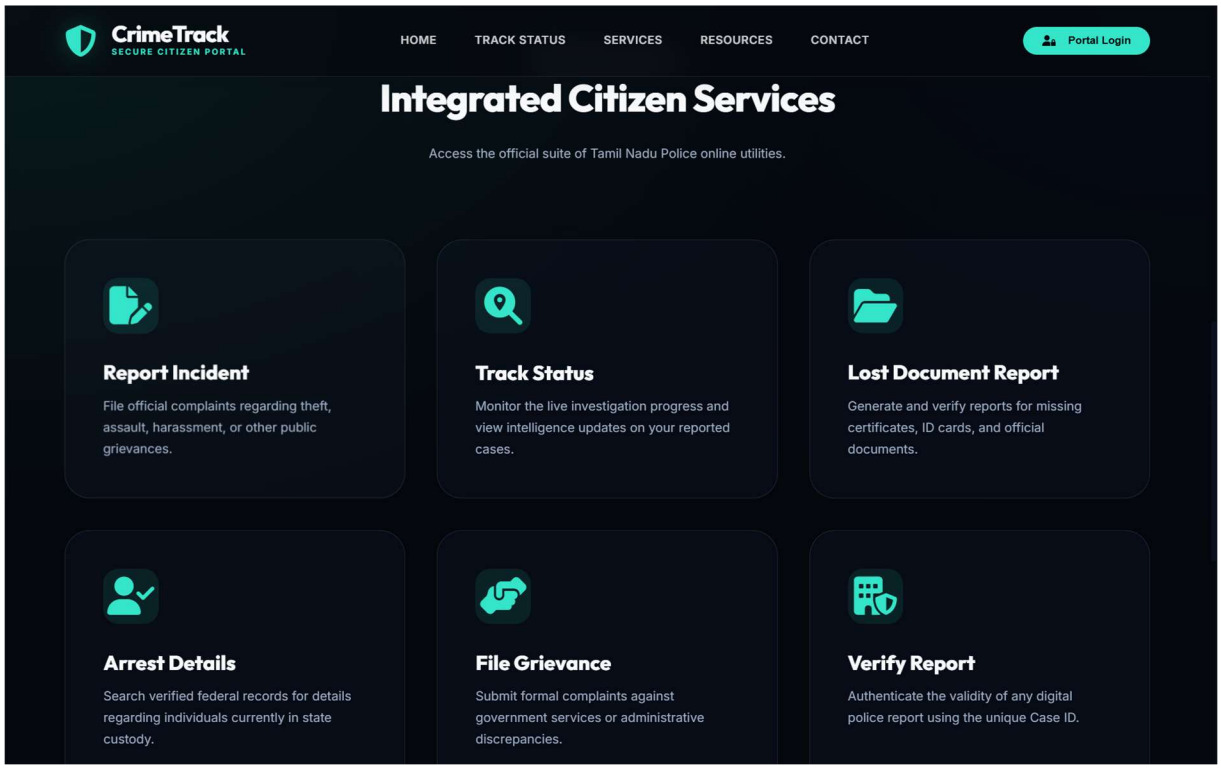
- Step 5: Draw business logic layer (Express API on App Service)
- Step 6: Draw data layer (MySQL on VM)
- Step 7: Add Azure DevOps pipeline as a separate track showing CI/CD flow
- Step 8: Draw arrows showing communication protocols (HTTPS, TCP/MySQL)
- Step 9: Add network boundary boxes (Internet, Azure VNET, Private Subnet)
- Step 10: Add security annotations — JWT, HTTPS, bcrypt, Firewall rules

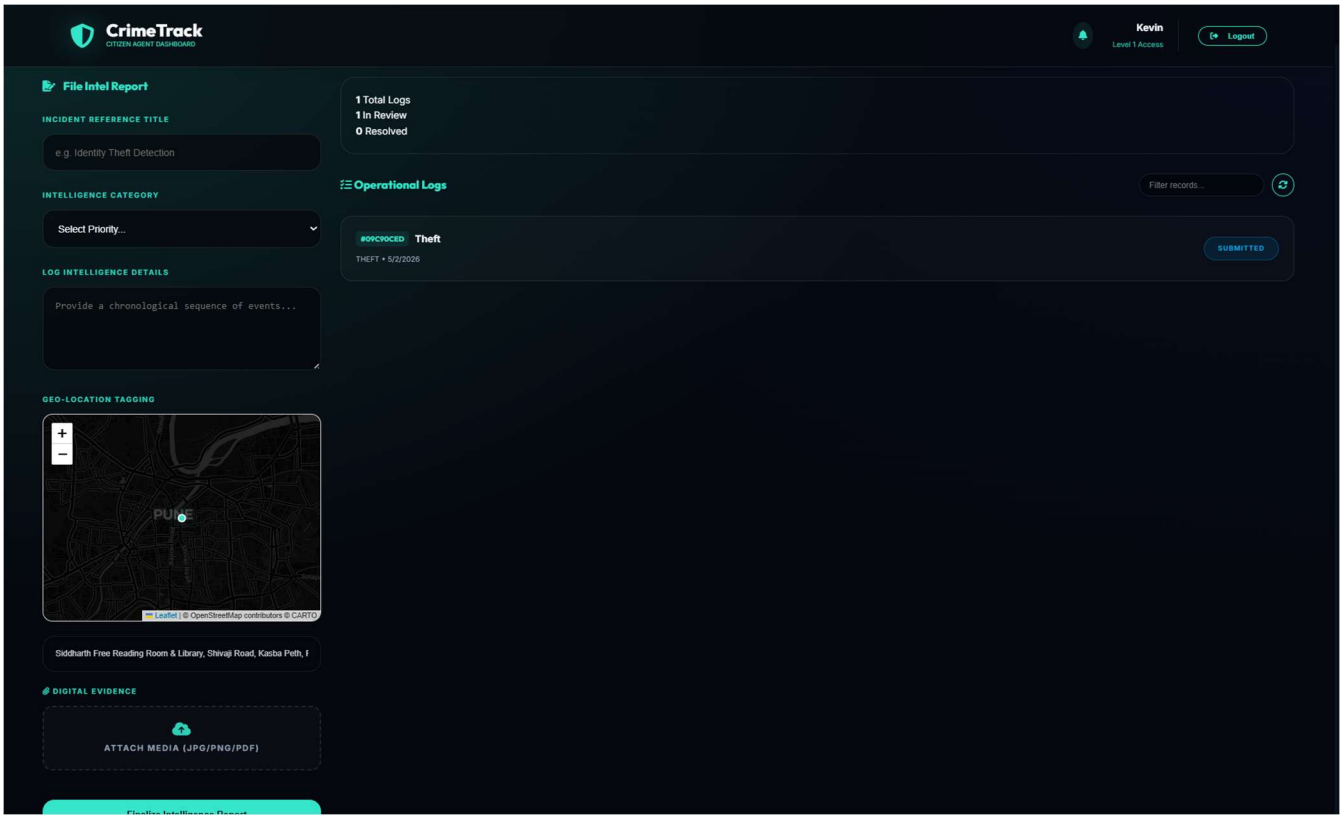


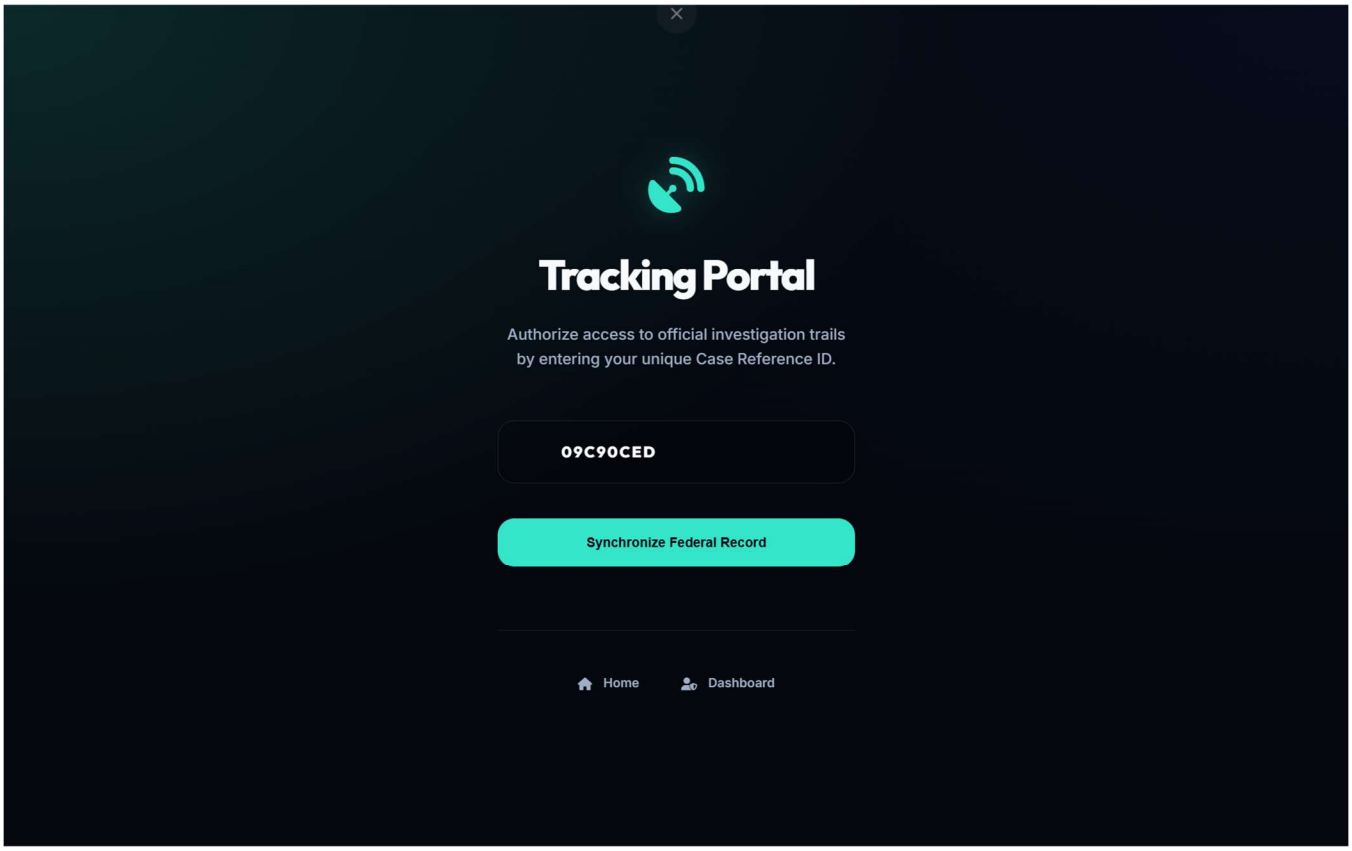
Experiment - 10

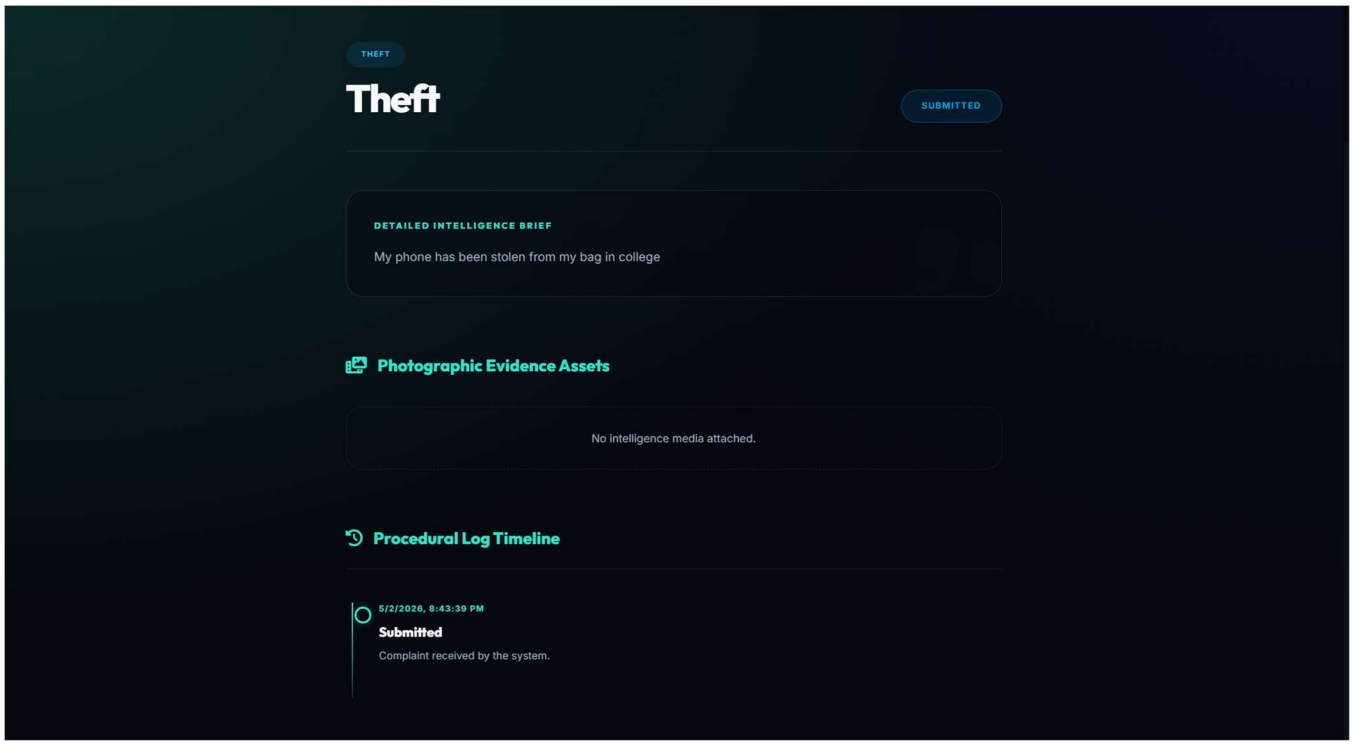
CrimeTrack UI

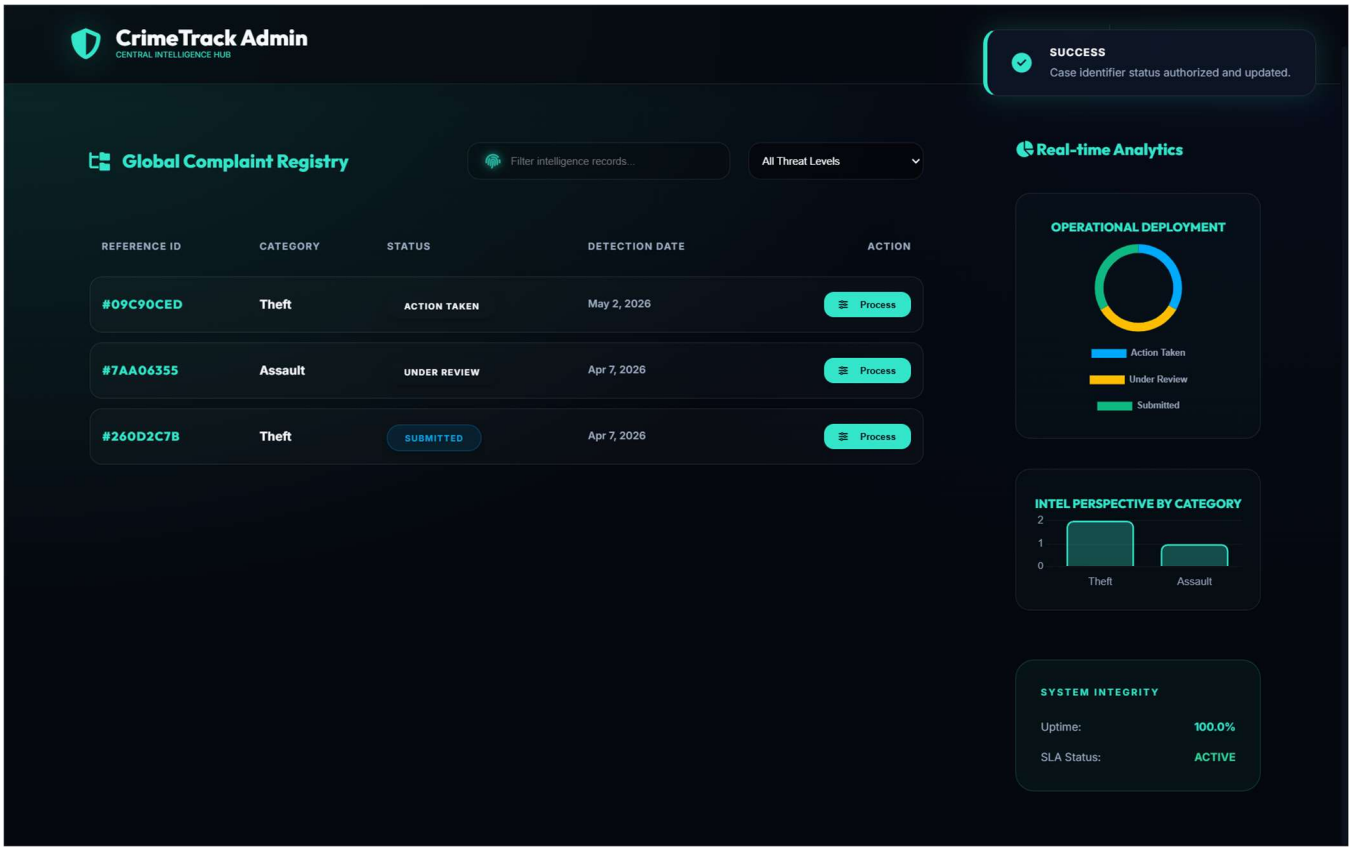





CrimeTrack
CITIZEN AGENT DASHBOARD
Kevin
Level 1 Access
Logout
File Intel Report
INCIDENT REFERENCE TITLE
e.g. Identity Theft Detection
INTELLIGENCE CATEGORY
Select Priority...
LOG INTELLIGENCE DETAILS
Provide a chronological sequence of events...
GEO-LOCATION TAGGING
PUNE
Leaflet | © OpenStreetMap contributors © CARTO
Siddharth Free Reading Room & Library, Shivaji Road, Kasba Peth, I
DIGITAL EVIDENCE
ATTACH MEDIA (JPG/PNG/PDF)
1 Total Logs
1 In Review
0 Resolved
Operational Logs
Filter records...
#09C90CED
Theft
THEFT • 5/2/2026
SUBMITTED


Tracking Portal
Authorize access to official investigation trails
by entering your unique Case Reference ID.
09C90CED
Synchronize Federal Record
Home
Dashboard


THEFT
Theft
SUBMITTED
DETAILED INTELLIGENCE BRIEF
My phone has been stolen from my bag in college
Photographic Evidence Assets
No intelligence media attached.
Procedural Log Timeline
5/2/2026, 8:43:39 PM
Submitted
Complaint received by the system.


CrimeTrack Admin
CENTRAL INTELLIGENCE HUB
SUCCESS
Case identifier status authorized and updated.
Global Complaint Registry
Filter intelligence records...
All Threat Levels
Real-time Analytics
REFERENCE ID
CATEGORY
STATUS
DETECTION DATE
ACTION
#09C90CED
Theft
ACTION TAKEN
May 2, 2026
Process
#7AA06355
Assault
UNDER REVIEW
Apr 7, 2026
Process
#260D2C7B
Theft
SUBMITTED
Apr 7, 2026
Process
OPERATIONAL DEPLOYMENT
Action Taken
Under Review
Submitted
INTEL PERSPECTIVE BY CATEGORY
2
1
0
Theft
Assault
SYSTEM INTEGRITY
Uptime:
100.0%
SLA Status:
ACTIVE

Experiment - 11

Implementation

Procedure:

Step 1: Set Up an Azure DevOps Project

- Log in to Azure DevOps.
- Click "New Project" → Enter project name → Click "Create".
- Inside the project, navigate to "Repos" to store the code.

Step 2: Add Your Web Application Code

- Navigate to Repos → Click "Clone" to get the Git URL.
- Open Visual Studio Code / Terminal and run:

  git clone <repo_url>

  cd <repo_folder>
- Add web application code (HTML, CSS, JavaScript, React, Angular, or backend like Node.js, .NET, Python, etc.).
- Commit & push:

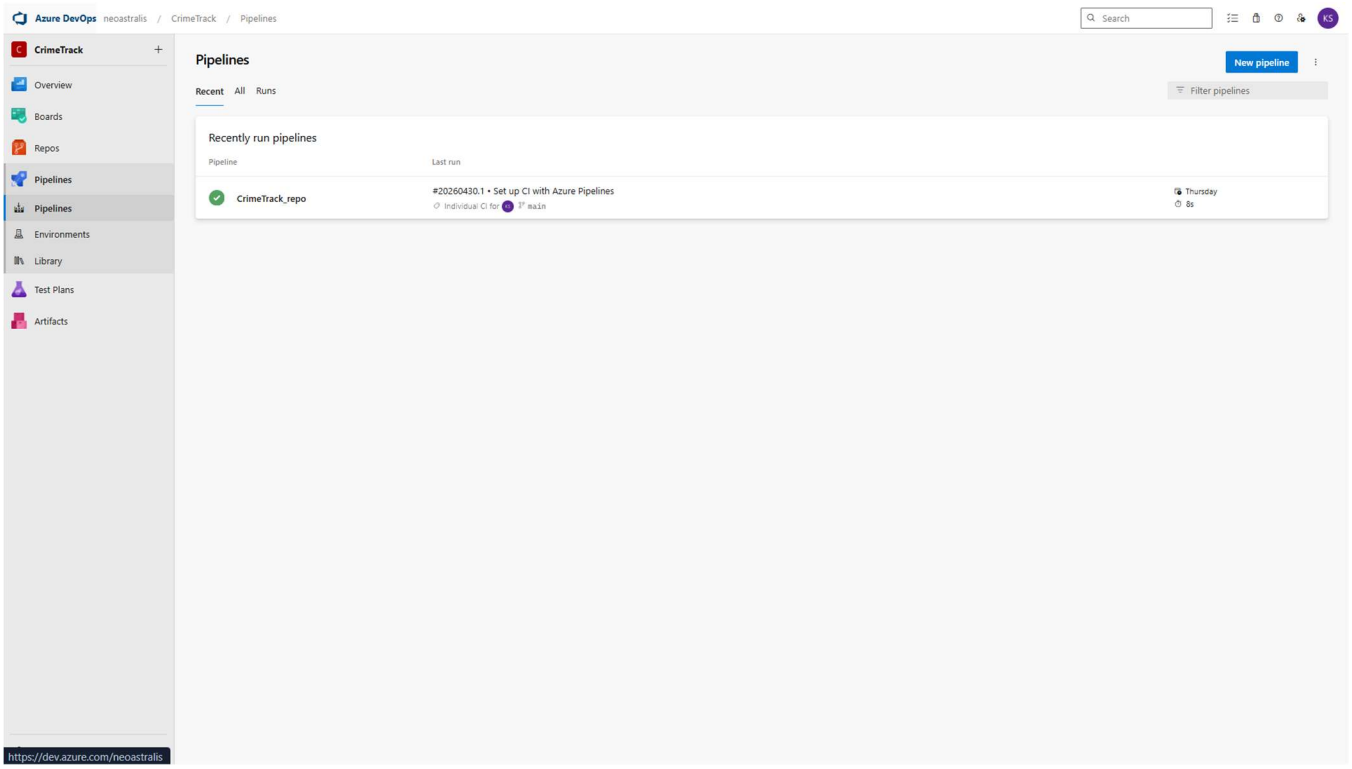
  git add .

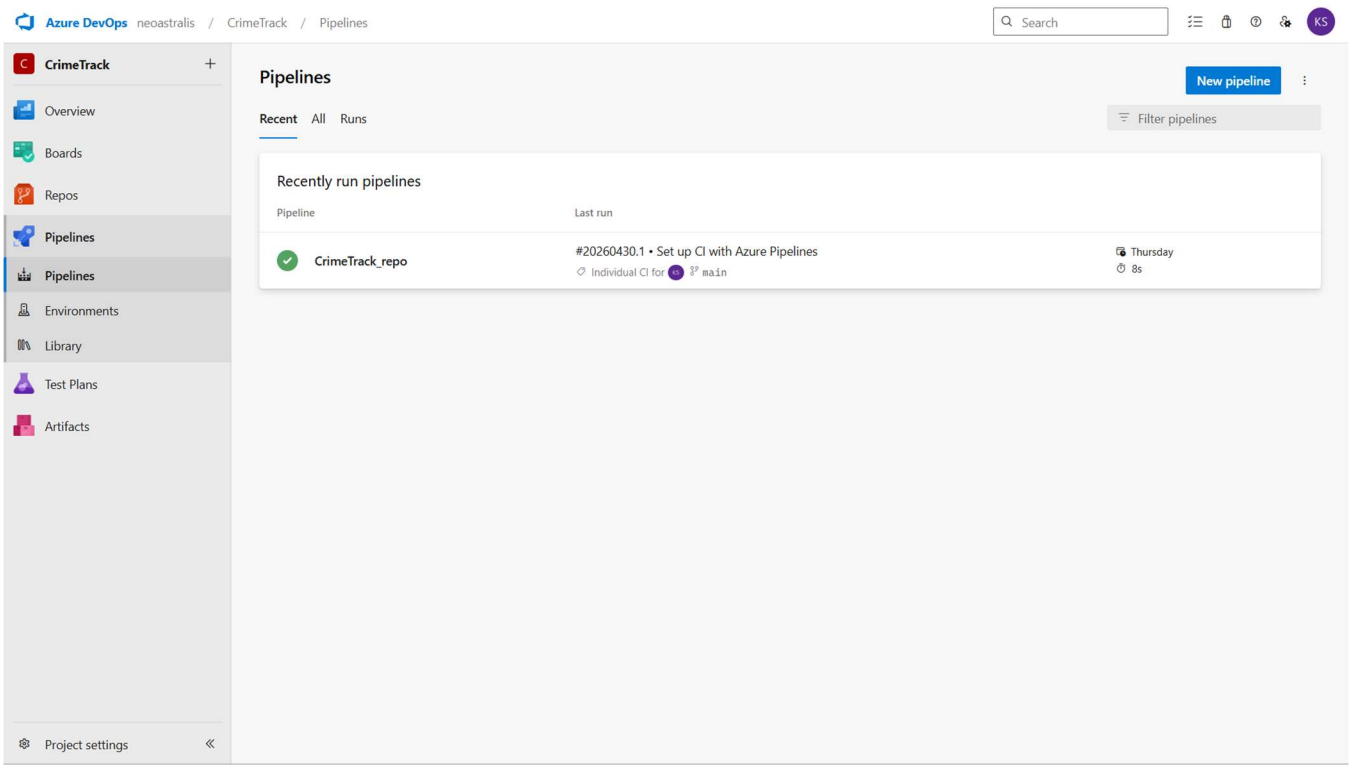
  git commit -m "Initial commit" git push origin main

Step 3: Set Up Build Pipeline (CI/CD - Continuous Integration)

- Navigate to Pipelines → Click "New Pipeline".
- Select Git Repository (Azure Repos, GitHub, or Bitbucket).
- Choose Starter Pipeline or a pre-configured template for your framework.
- Modify the azure-pipelines.yml file (Example for a Node.js app):

Click "Save and Run" → The pipeline will start building app.


Azure DevOps neoastralis / CrimeTrack / Pipelines
Search
KS
CrimeTrack
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
Pipelines
New pipeline
Recent All Runs
Filter pipelines
Recently run pipelines
Pipeline
Last run
CrimeTrack_repo
#20260430.1 • Set up CI with Azure Pipelines
Individual CI for KS main
Thursday
8s
https://dev.azure.com/neoastralis

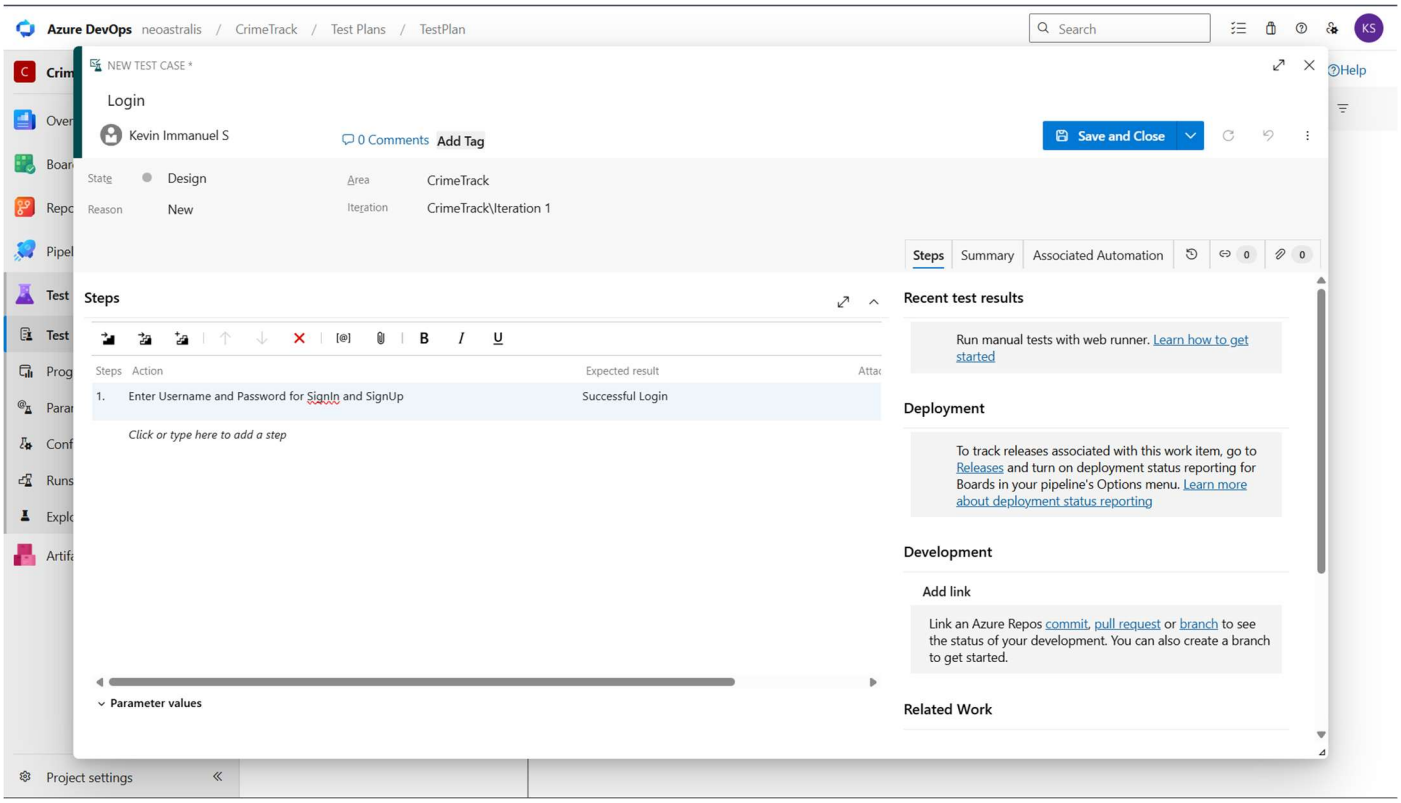

Azure DevOps neoastralis / CrimeTrack / Pipelines
Search
KS
CrimeTrack
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
Project settings
Pipelines
New pipeline
Recent All Runs
Filter pipelines
Recently run pipelines
Pipeline
Last run
CrimeTrack_repo
#20260430.1 • Set up CI with Azure Pipelines
Individual CI for KS main
Thursday
8s

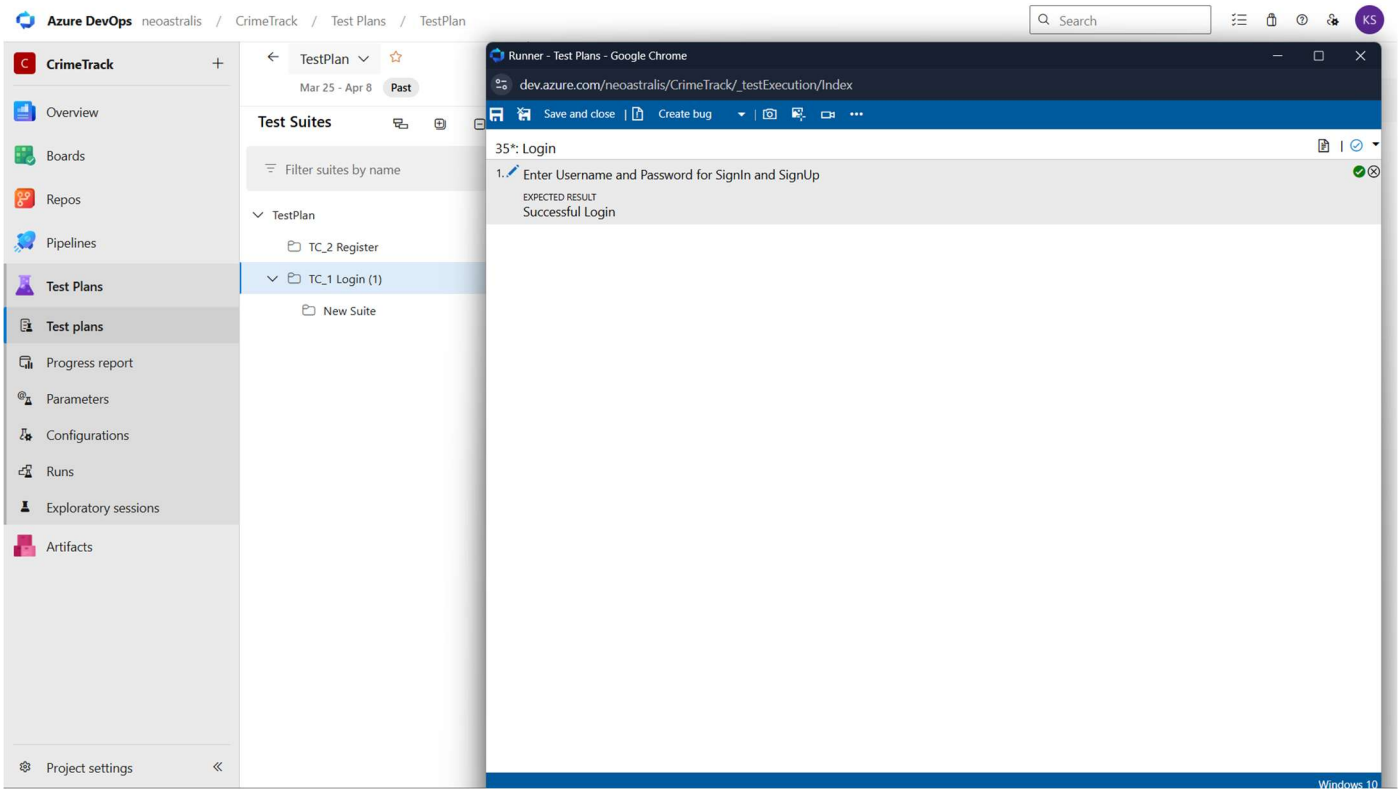
Experiment - 12

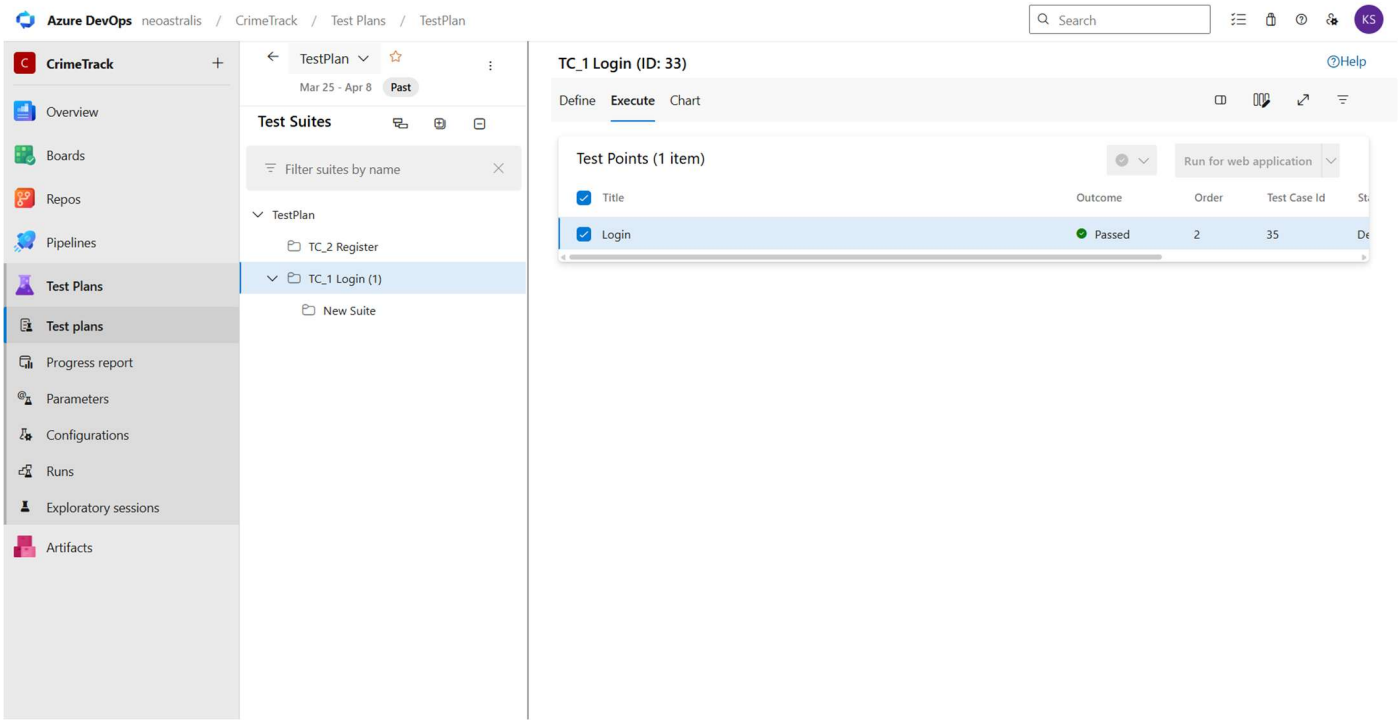
Testing

Procedure:

1. Go to Azure DevOps → Select Test Plans.
2. Click "New Test Plan" → Add test cases.
3. Assign tests to team members for execution.
4. Track results using Test Runs & Reports




Azure DevOps neoastralis / CrimeTrack / Test Plans / TestPlan
Search
KS
CrimeTrack
Overview
Boards
Repos
Pipelines
Test Plans
Test plans
Progress report
Parameters
Configurations
Runs
Exploratory sessions
Artifacts
Project settings
TestPlan
Mar 25 - Apr 8
Past
Test Suites
Filter suites by name
TestPlan
TC_2 Register
TC_1 Login (1)
New Suite
Runner - Test Plans - Google Chrome
dev.azure.com/neoastralis/CrimeTrack/_testExecution/Index
Save and close
Create bug
35*: Login
1. Enter Username and Password for SignIn and SignUp
EXPECTED RESULT
Successful Login
Windows 10


Azure DevOps neoastralis / CrimeTrack / Test Plans / TestPlan
Search
KS
CrimeTrack
Overview
Boards
Repos
Pipelines
Test Plans
Test plans
Progress report
Parameters
Configurations
Runs
Exploratory sessions
Artifacts
TestPlan
Mar 25 - Apr 8
Past
Test Suites
Filter suites by name
TestPlan
TC_2 Register
TC_1 Login (1)
New Suite
TC_1 Login (ID: 33)
Help
Define
Execute
Chart
Test Points (1 item)
Run for web application
Title
Outcome
Order
Test Case Id
Login
Passed
2
35